



UNIVERSIDAD
DE GRANADA

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Simulador de fuego en tiempo real para sistemas portátiles en Godot 4.x

Autor

Gerardo Bonet Pérez

Director

Germán Arroyo Moreno



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2026

Simulador de fuego en tiempo real para sistemas portátiles en Godot 4.x

Gerardo Bonet Pérez

Palabras clave: Incendios forestales, Godot 4, Autómatas Celulares, Raymarching, Renderizado Volumétrico, Compute Shaders, GPGPU, Modelo Volumétrico, Shaders

Resumen

La simulación precisa e inmersiva de incendios forestales en tiempo real supone un desafío crítico para la prevención y la toma de decisiones operativas durante las emergencias medioambientales. Sin embargo, sigue habiendo una gran brecha tecnológica entre los simuladores de alto rigor científico, que carecen de representaciones tridimensionales inmersivas, y las aproximaciones desde los motores gráficos, que tienen una gran fidelidad visual pero sin seguir un rigor termodinámico.

Para resolver esta brecha, en este Trabajo de Fin de Grado se presenta un simulador de incendios interactivo construido en el motor Godot 4 que unifica la precisión de los modelos de propagación científicos con el renderizado visual. La solución propuesta delega el cálculo de la propagación, basado en las ecuaciones de Rothermel, a la GPU aprovechando su paralelismo mediante Autómatas Celulares y Compute Shaders. A su vez, el sistema visualiza el volumen del fuego calculado por la GPU sobre Modelos Digitales de Elevación (DEM) reales utilizando un algoritmo de Raymarching interactivo, uniendo así los dos ámbitos.

Los experimentos de evaluación demuestran que el sistema replica el comportamiento físico del fuego ante las distintas variaciones topográficas y medioambientales, logrando procesar matrices volumétricas masivas manteniendo un rendimiento superior a los 150 fotogramas por segundo. Este enfoque integrado representa un avance tecnológico muy prometedor para la creación de herramientas portátiles de apoyo a la toma de decisiones en la gestión de incendios forestales.

Real-Time Fire Simulator for Portable Systems in Godot 4.x

Gerardo Bonet Pérez

Keywords: Wildfire, Godot 4, Cellular Automata, Raymarching, Volumetric Rendering, Compute Shaders, GPGPU, Volumetric Model, Shaders

Abstract

Accurate and immersive real-time simulation of wildfires poses a critical challenge for prevention and operational decision-making during environmental emergencies. However, there remains a significant technological gap between simulators with high scientific rigor, which lack immersive three-dimensional representations, and graphics engine approaches, which offer high visual fidelity but lack thermodynamic rigor.

To bridge this gap, this Bachelor's Degree Final Project presents an interactive fire simulator built in the Godot 4 engine that unifies the precision of scientific propagation models with visual rendering. The proposed solution delegates the calculation of fire spread, based on Rothermel's equations, to the GPU, leveraging its parallelism through Cellular Automata and Compute Shaders. Simultaneously, the system visualizes the fire volume calculated by the GPU over real Digital Elevation Models (DEM) using an interactive Raymarching algorithm, thus uniting both fields.

Evaluation experiments demonstrate that the system replicates the physical behavior of fire under various topographical and environmental variations, successfully processing massive volumetric matrices while maintaining a performance of over 150 frames per second. This integrated approach represents a highly promising technological advancement for the creation of portable decision-support tools in forest fire management.

Yo, **Gerardo Bonet Pérez**, alumno de la titulación **TITULACIÓN de la Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 20887449-E, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Gerardo Bonet Pérez

Granada, 4 de septiembre de 2026

Agradecimientos

En primer lugar, me gustaría agradecer a mi tutor, Germán, por darme la oportunidad y por su constante implicación y paciencia durante todo el desarrollo de este Trabajo de Fin de Grado. Gracias a su orientación y sus valiosos consejos que me han guiado y ayudado a afrontar los distintos retos del proyecto con gran seguridad.

También quiero dar las gracias a todos esos amigos que me han acompañado en los años de estudio en ETSIIT, con quienes he compartido interminables horas de clases y de estudio pero también de risas y buenos recuerdos. Su compañía me ha hecho mucho más llevadero el día a día de la carrera, convirtiendo esta etapa en una experiencia inolvidable.

Por último, el agradecimiento más profundo es para mi familia, en especial a mis padres, por el apoyo incondicional en el día a día, dándome la confianza que necesitaba para seguir adelante y poder llegar hasta este punto.

A todos vosotros, gracias por formar parte de este camino.

Índice general

1. Introducción	9
1.1. Motivaciones	14
1.2. Objetivos	16
1.3. Planificación	18
1.3.1. Diagrama de Gantt	20
2. Estado del Arte y Marco Teórico	21
2.1. Fundamentos de la Informática Gráfica	21
2.2. Renderizado Volumétrico	24
2.3. Modelos Físicos de Propagación	27
2.3.1. Autómatas Celulares	29
2.3.2. Método de Monte Carlo	31
3. Metodología	34
3.1. Selección de herramientas	34
3.1.1. Comparativa de motores gráficos	35
3.2. Arquitectura del Simulador	37
3.2.1. <i>Pipeline</i> Gráfico	37
3.2.2. <i>Pipeline</i> de Cómputo	39
3.2.3. El motor de Simulación	40
3.2.4. Visualización	44
3.2.5. Sincronización CPU-GPU	48
3.3. Topografía	50
3.3.1. Procesamiento de datos	50
3.3.2. Renderizado del terreno	51
3.3.3. Construcción de la matriz lógica	52
3.4. Interacción y Control Dinámico	53
3.4.1. Generación de colisiones	53
3.4.2. Navegación espacial	53
3.4.3. Sistema de Ignición	53
3.4.4. Interfaz de Usuario	54
3.5. Desafíos Técnicos y Soluciones	55
3.5.1. Optimización del <i>Raymarching</i>	55

3.5.2. Sincronización de Espacios de Coordenadas	56
3.5.3. Anomalías en el Relieve	56
3.5.4. Desincronización de Cámaras	57
3.6. Integración y uso de Inteligencia Artificial	58
4. Resultados y Validación	59
4.1. Entorno y Metodología de Evaluación	59
4.2. Validación del Modelo Físico	60
4.3. Evaluación del rendimiento	63
5. Conclusiones	67
5.1. Posibles líneas y mejoras futuras de trabajo	68
Referencias	71

Índice de figuras

1.1. Nube de puntos LiDAR	10
1.2. Malla de triángulos 3D	11
1.3. Proceso de Raymarching	13
2.1. Proceso de Rasterización	22
2.2. Etapas principales del Pipeline Gráfico	22
2.3. Formación de MarchingCubes	25
2.4. Trazado de Rayos	26
2.5. Ejemplo de mapa de calor 2D de un incendio	33
3.1. Vecindad de Moore	42
3.2. Caras traseras obtenidas por el FBO	45
3.3. Esquema de nodos de la escena	46
3.4. Mapa generado por un DEM	52
3.5. Interfaz de Usuario del simulador	55
4.1. Propagación sin obstaculos	60
4.2. Propagación con influencia eólica	61
4.3. Propagación con influencia topográfica	62
4.4. Simulación con mapa de elevaciones	63
4.5. Gráfica de rendimiento según la resolución	64
4.6. Gráfica de evolución de FPS	66

Índice de tablas

2.1. Valores empíricos para el Autómata Celular	31
3.1. Comparativa técnica de motores gráficos	36
4.1. Evolución del rendimiento frente a la resolución	64
4.2. Evolución del rendimiento durante la propagación	65

Capítulo 1

Introducción

El progresivo abandono del medio rural y la falta de gestión forestal durante las últimas décadas han provocado una acumulación descontrolada de biomasa en nuestros montes, a lo cual se le suma las variaciones de temperatura derivadas del cambio climático, dando lugar a una nueva generación de incendios forestales mucho más extremos, rápidos e impredecibles. Hoy en día, estos eventos representan uno de los mayores retos a nivel ecológico y de seguridad ciudadana, haciendo que la necesidad de predecir su comportamiento y avance sea más crítica que nunca.

Para hacer frente a este desafío, la tecnología se ha convertido en el mejor aliado de la gestión de emergencias, y es aquí donde la **Informática Gráfica** cobra un papel protagonista. Aunque tradicionalmente este campo de las ciencias de la computación es conocido por la creación de imágenes para el ocio y los videojuegos, su verdadera potencia radica en su capacidad para modelar matemáticamente el mundo real. Utilizando algoritmos complejos, álgebra y cálculo espacial para ubicar objetos y la óptica física para calcular la iluminación, los sistemas gráficos actuales permiten transformar datos abstractos en simulaciones inmersivas que ayudan a predecir dinámicas complejas [1].

A nivel funcional, la informática gráfica se divide en dos grandes bloques que resultan fundamentales para construir un simulador forestal: el **modelado 3D**, que define la estructura matemática de los elementos, y el **renderizado**, encargado de calcular la luz y generar la imagen final. Sin embargo, para que un simulador forestal tenga utilidad real, no basta con generar gráficos creíbles; el fuego virtual debe interactuar con una topografía idéntica a la del mundo físico.

Para lograr este nivel de precisión en el modelado del terreno, la informática gráfica se apoya en los Sistemas de Información Geográfica (SIG)

y la teledetección. Los datos se capturan habitualmente mediante sensores LiDAR (*Light Detection and Ranging*), los cuales emiten miles de pulsos láser desde el aire, atravesando la vegetación para impactar contra el suelo y generar una nube de puntos 3D masiva. A partir del filtrado de estos puntos se obtienen los Modelos Digitales de Elevación (MDE), matrices bidimensionales donde cada píxel almacena la altitud exacta, proporcionando así el relieve o *heightmap* sobre el que interactuarán los modelos físicos [2][3].

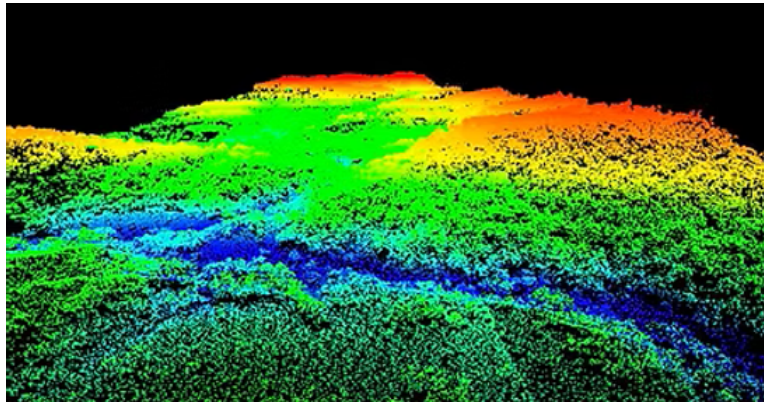


Figura 1.1: Coloración por elevación de nube de puntos LiDAR

Una vez construido el escenario topográfico, el siguiente reto es simular el comportamiento físico de las llamas. Históricamente, la predicción de este avance se abordó mediante modelos matemáticos y empíricos, siendo el estándar principal el modelo de Rothermel (1972) [4]. Este modelo matemático se basa en el principio de conservación de la energía para calcular la velocidad de propagación en un punto concreto, basándose en la carga de combustible, la inclinación del terreno y factores ambientales.

Sin embargo, las ecuaciones de Rothermel calculan el comportamiento del fuego de forma unidimensional y bajo condiciones uniformes, quedándose lejos de cómo evoluciona un incendio en la vida real. Para simular la expansión de un incendio sobre complejas superficies geográficas, la metodología evolucionó hacia modelos de propagación bidimensionales, donde destacaron los modelos vectoriales basados en el principio de Huygens [5], que modelan el frente del fuego como un conjunto de elipses continuas en expansión. La solución que aportan dichos modelos, aunque es precisa, resulta computacionalmente muy pesados a la hora de calcular intersecciones y resolver cambios topográficos bruscos en tiempo real [6, 7].

Ante estas desventajas se buscó una alternativa algorítmica más eficiente, llevando a la simulación espacial a evolucionar hacia la discretización del

entorno mediante sistemas de autómatas celulares. Este enfoque divide el terreno en una matriz de celdas donde cada una de ellas posee un estado térmico e información crucial acerca del combustible, para así siguiendo una serie de reglas de transición el autómata sea capaz de calcular exactamente en qué momento una celda contagia a la siguiente. Al superponer esta matriz sobre los mapas de alturas comentados anteriormente, el autómata es capaz de simular fenómenos físicos de forma natural y de manera más eficiente integrándose con las arquitecturas gráficas modernas.

Finalmente, el último gran desafío de la informática gráfica en este campo es la fase de renderizado: ¿cómo visualizamos este fenómeno de manera realista? Tradicionalmente, en la informática gráfica, se ha optado por el modelado mediante mallas, que consisten en una estructura formada por vértices colocados en el espacio y que se unen mediante aristas creando polígonos (generalmente triángulos) que acaban dando forma a un modelo el cual está vacío en su interior como el que se muestra en la figura 1.2. Posteriormente la renderización de estas mallas se realiza mediante la rasterización, que es el proceso algorítmico que proyecta estos triángulos tridimensionales sobre un plano bidimensional, pudiéndose representar así en los píxeles de una pantalla [8].

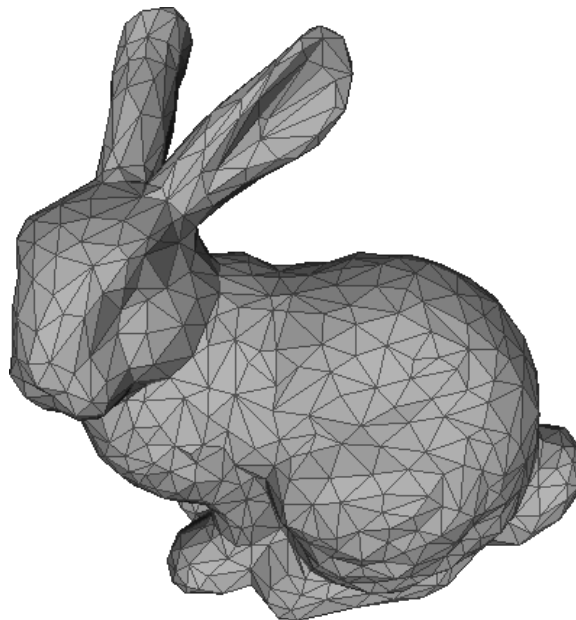


Figura 1.2: Ejemplo de una malla poligonal de triángulos superficiales

Este enfoque es excelente para renderizar objetos con límites bien de-

finidos como montañas sólidas, pero fracasa rotundamente con fenómenos amorfos y caóticos como el fuego o el humo, que carecen de una superficie sólida y estática. Esto obliga a abandonar las mallas y adoptar el **modelado volumétrico** mediante vóxeles, que utiliza una estructura tridimensional de celdas para representar no solo la frontera, sino el interior de un modelo, donde cada vóxel almacena propiedades físicas continuas, como la densidad, la temperatura o la concentración de combustible.

Al igual que cambia el modelado el algoritmo de renderizado también cambia cuando nos referimos a modelos volumétricos como el del fuego y pasamos de usar la rasterización a implementar técnicas de Renderizado Volumétrico Directo (DVR). A diferencia de los métodos indirectos que extraen mallas poligonales intermedias para representar la capa exterior de un volumen, el DVR permite procesar y visualizar toda la información de forma directa sin depender de geometrías exactas [9].

Existen varias técnicas de DVR que, aunque fueron pioneras, presentan limitaciones para simulaciones dinámicas:

- **Splatting (Proyección de huellas):** En esta técnica, el algoritmo procesa el volumen proyectando cada vóxel individualmente sobre el plano de la imagen. A medida que estas huellas se van acumulando en la pantalla, sus colores y transparencias se suman para formar la imagen final. Aunque es un método rápido, presenta problemas de solapamiento y tiende a generar imágenes borrosas cuando se mueve la cámara [10].
- **Renderizado basado en texturas (Texture-based Volume Rendering):** Este método divide el volumen en planos que se alinean de forma paralela a la cámara y se les asigna la información de los vóxeles como si fuera una textura 2D. Tienen dos problemas principales, por una parte si se rota la cámara todo el sistema de planos debe recalcularse para volver a alinearse con esta, y por otro lado un número bajo de láminas produce un efecto visual escalonado mientras que un número alto dispara el consumo de memoria [11].

Debido a estos problemas, las dos técnicas podrían ser consideradas para la representación de volúmenes estáticos, pero para simulaciones dinámicas como las de un incendio forestal no serían lo suficientemente flexibles.

Para el desarrollo de estos y poder aprovechar al máximo la capacidad de procesamiento paralelo de las tarjetas gráficas modernas, la técnica elegida para este simulador es el trazado de rayos volumétrico (*Raymarching*). Este algoritmo lanza rayos virtuales desde la cámara hacia la cuadrícula 3D, avanzando a través del volumen en pequeños pasos, donde en cada uno de

ellos evalúa los datos del vóxel y calcula cómo la luz se acumula, se bloquea o se dispersa al atravesarlo [12].

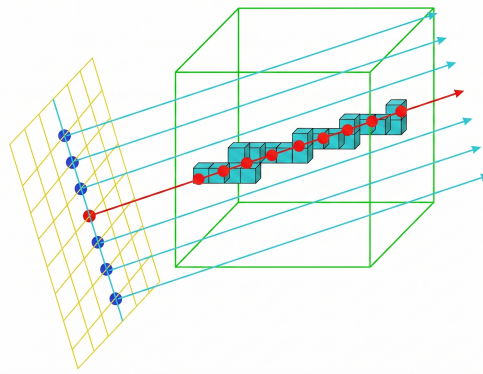


Figura 1.3: Proceso de Raymarching

Además de la visualización de los volúmenes, en un simulador hay que añadirle el coste computacional del cálculo de las físicas continuas de la propagación, que requiere resolver complejas ecuaciones por cada voxel. Realizar esta cantidad de cálculos en la unidad central de procesamiento (CPU) resultaría prácticamente imposible y saturaría el simulador haciéndolo inservible.

Es a partir de aquí donde coge toda la prioridad la evolución de las Unidades de Procesamiento Gráfico (GPU) las cuales, a parte de dedicarse a la generación de imágenes, las GPU más modernas permiten la ejecución de programas de propósito general denominados *Compute Shaders*. Estos programas consiguen aprovechar los miles de núcleos de la tarjeta gráfica para así paralelizar cálculos matemáticos masivos, que permiten resolver toda la lógica de un autómata celular y las físicas implicadas en una fracción del tiempo que requeriría una CPU [13, 14].

Bajo este contexto, en el presente trabajo se desarrollará un sistema que combinará el modelado topográfico a partir de datos geográficos reales (MDE y LiDAR) para construir un entorno navegable sobre el que se aplicará un modelo físico de propagación de incendios, aprovechando la potencia de la GPU mediante un *Compute Shaders* con autómatas celulares para una mayor eficiencia. Finalmente, la visualización de dicho entorno se llevará a cabo utilizando técnicas de renderizado volumétrico directo (*Raymarching*) en tiempo real.

1.1. Motivaciones

El modelado volumétrico se ha convertido en un pilar fundamental en varios ámbitos científicos ya que permite tener representaciones detalladas del interior de las estructuras. Esto destaca en áreas como la medicina, donde la reconstrucción tridimensional de tomografías y resonancias permite el diagnóstico no invasivo, o en la ingeniería y aerodinámica, donde es imprescindible para analizar el comportamiento de los fluidos. En la actualidad, gracias a la evolución de las GPU y su gran capacidad de paralelización es posible procesar y renderizar estos volúmenes dinámicos en tiempo real abriendo la puerta al desarrollo de simuladores científicos que no necesitan un hardware altamente especializado para su ejecución.

Uno de los sectores que necesita adoptar este salto tecnológico con mayor urgencia es en el ámbito medioambiental, más concretamente la prevención y gestión de incendios forestales. Se ha podido observar que en los últimos años ha cambiado el clima en la península Ibérica, provocando que cada vez ocurran incendios más frecuentemente con fuegos mucho más impredecibles, haciendo que cada vez sea más compleja su extinción minimizando las pérdidas ecológicas y humanas.

Para hacer frente a este cambio, resultaría de gran ayuda que los equipos de emergencia tengan a sus disposición herramientas de simulación que ofrezcan predicciones precisas, pero hasta el momento, la simulación de estos desastres se ha apoyado principalmente en modelos bidimensionales (2D) sobre Sistemas de Información Geográfica (SIG). Si bien, los modelos matemáticos de estos simuladores procesan toda la topografía del terreno y los factores medioambientales para predecir la propagación correctamente, su representación en mapas 2D hace que sea más compleja su comprensión obligando al operador a interpretar el relieve a través de curvas de nivel o mapas de color.

La implementación de un entorno visual tridimensional (3D) elimina este problema facilitando a los operativos de emergencias una rápida comprensión del estado actual del incendio. Al renderizar el relieve con su verdadera escala y volumen les proporciona una percepción directa de la orografía identificando así rápidamente futuros corredores topográficos y puntos críticos para mejorar la toma de decisiones tácticas y planificación de la extinción.

Por concluyente, la motivación principal de este trabajo es proponer una solución que combine la precisión de modelos matemáticos de propagación, como el estándar de Rothermel, con un entorno gráfico tridimensional que aproveche los avances en tecnología de la GPU. Para lograr esta interacción entre el cálculo de las físicas y la ejecución en tiempo real sin que se vea afectado el rendimiento se hará uso de la técnica de renderizado de volumen directo(*Raymarching*) junto con *Compute Shaders* para la paralelización de cálculos masivos. El resultado final esperado es una herramienta donde se junte los modelos tradicionales con una representación espacial interactiva, facilitando así la toma de decisiones estratégicas durante una emergencia.

1.2. Objetivos

El objetivo principal de este proyecto es diseñar e implementar un simulador de incendios forestales en tiempo real mediante la técnica de renderizado de *Raymarching* y autómatas celulares dentro del motor gráfico Godot. El fin es proporcionar una herramienta interactiva que fusione modelos matemáticos precisos de propagación (como el de Rothermel) con técnicas de renderizado avanzado, superando las limitaciones visuales de los sistemas 2D clásicos.

Para alcanzar este objetivo, se establecen los siguientes objetivos:

1. **Procesar datos geográficos reales para la generación de la topografía 3D.**

Este objetivo consiste en la obtención y tratamiento de Modelos Digitales de Elevación (MDE), para convertirlos en mapas de alturas (*heightmaps*) que serán usados para generar el relieve tridimensional sobre el que interactuará el modelo del incendio.

2. **Aprender a utilizar el entorno de Godot 4, GDScript y su lenguaje de *shaders* visuales.**

Este objetivo se centra en familiarizarse con la arquitectura general del motor y dominar su lenguaje de *scripting* (GDScript) para la gestión de la aplicación, la interfaz y la interacción con el usuario, además de profundizar en el lenguaje de *shaders* propio de Godot para la representación gráfica y la programación del algoritmo de renderizado.

3. **Investigar e integrar *Compute Shaders* para cálculos de propósito general (GPGPU).**

Este objetivo es crítico para el rendimiento del proyecto, consiste en aprender a desarrollar subprogramas de cálculo puro (*Compute Shaders*) y comprender cómo se sincroniza el flujo de datos entre la CPU y la GPU dentro del entorno de Godot estableciendo la infraestructura necesaria para el cálculo masivo

4. **Implementar la lógica matemática de propagación mediante autómatas celulares .**

Una vez establecida la infraestructura GPGPU, este objetivo se centra traducir las reglas físicas del fuego y los algoritmos estocásticos a un sistema de autómatas celulares, donde la lógica de actualización celda a celda se programará dentro del *Compute Shader*, permitiendo procesar el avance del incendio de forma paralela y en tiempo real.

5. Diseñar e implementar el renderizado volumétrico mediante *Raymarching*.

Para poder representar visualmente los datos calculados en el paso anterior es necesario construir un algoritmo de renderizado directo el cual se programará en un *shader* espacial que seguirá la técnica de *Raymarching* para visualizar el humo y el fuego en función de los valores térmicos que el autómata celular actualiza en tiempo real.

6. Integrar los sistemas en un entorno interactivo y controlable por el usuario.

El objetivo es unificar la topografía, el autómata celular y el algoritmo de *Raymarching* en una única aplicación desarrollando una interfaz interactiva que permita al usuario configurar las variables iniciales de la simulación, como el foco de ignición o los factores ambientales, y observar la simulación en tiempo real.

7. Evaluar el rendimiento computacional y la estabilidad de la simulación.

Se trata de analizar y medir la solución desarrollada sometiendo al simulador a diferentes escenarios y pruebas de carga con el fin de monitorizar el consumo general de los recursos de hardware y la fluidez del renderizado. De este modo, se pretende demostrar que la solución técnica propuesta es escalable y viable como herramienta interactiva.

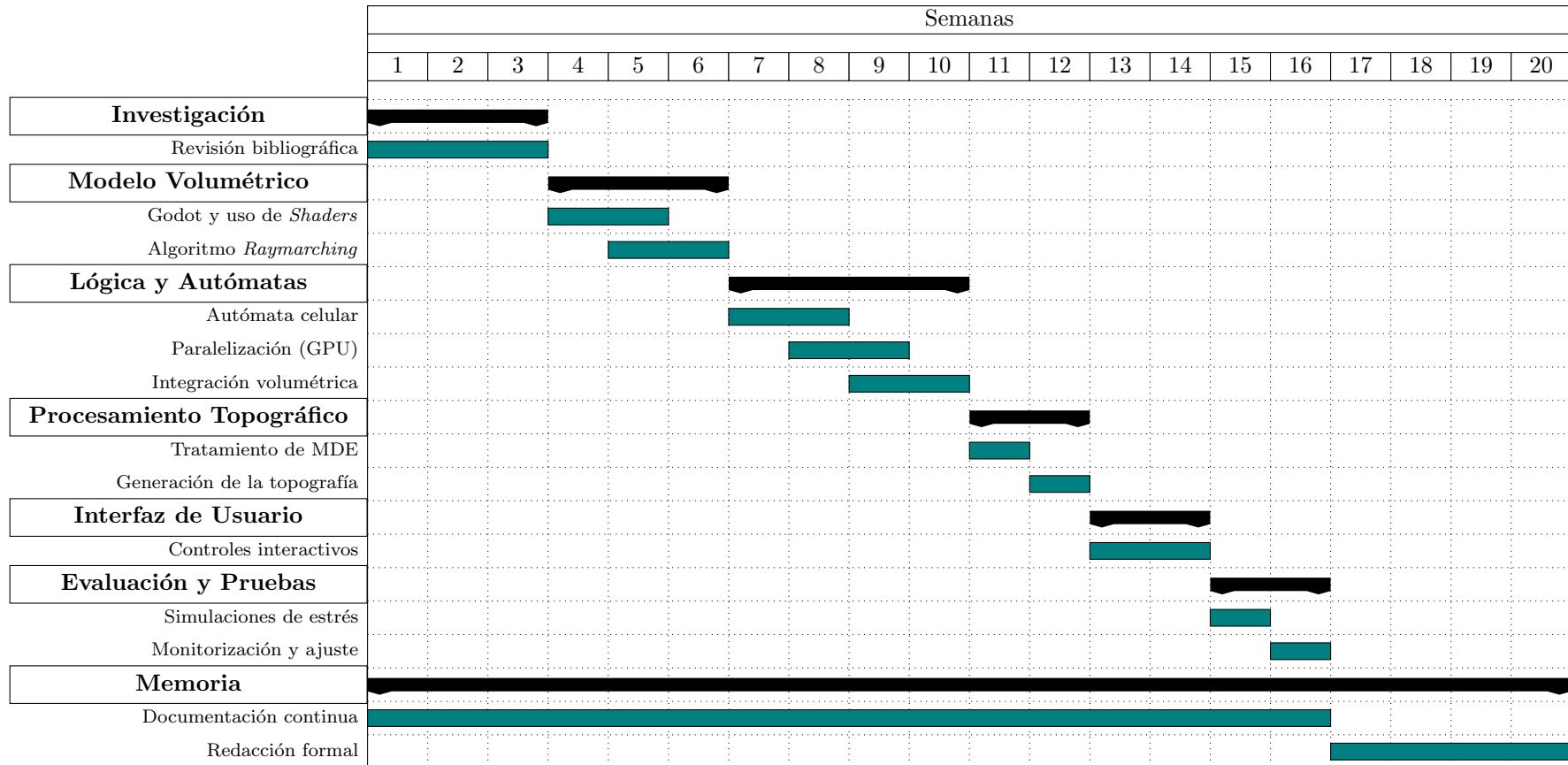
1.3. Planificación

Este trabajo se ha planificado siguiendo una metodología iterativa e incremental, lo que permite el desarrollo de las diferentes fases de la realización de este proyecto de manera estructurada. A continuación, se justifica la distribución temporal de las actividades, estimando una duración aproximada para cada una de ellas:

- **Investigación y Estado del Arte (3 semanas):** Antes de comenzar es necesario realizar una investigación sobre los conceptos matemáticos y las tecnologías gráficas relacionadas para comprender las limitaciones de los sistemas actuales y definir la arquitectura a implementar en Godot 4.
- **Creación del modelo volumétrico (3 semanas):** Una vez establecida la base teórica, se comenzará con la primera fase práctica que consiste en el desarrollo del algoritmo de visualización mediante el uso del lenguaje de *shaders* de Godot. El objetivo es generar un sistema capaz de interpretar texturas o matrices 3D para generar una representación volumétrica realista del fuego y el humo.
- **Implementación lógica mediante autómatas celulares (4 semanas):** Una vez que el sistema sea capaz de renderizar volúmenes y terrenos, se desarrollará el autómata celular encargado de calcular la propagación utilizando *Compute Shaders* para su paralelización en GPU. El punto crítico de esta fase será lograr conectar dinámicamente los datos resultantes de este cálculo con el modelo volumétrico creado en la etapa anterior.
- **Procesamiento de modelos topográficos y de elevación (2 semanas):** Hasta este punto toda la estructura y desarrollo se ha realizado sobre un mapa simple generado para hacer pruebas y es a partir de este momento cuando se procederá a la obtención y tratamiento de Modelos Digitales de Elevación (MDE). En esta fase se lleva a cabo el filtrado de estos datos geográficos y su exportación a mapas de alturas (*heightmaps*) para poder generar el relieve tridimensional.
- **Desarrollo de la interfaz de usuario (2 semanas):** Con los motores físico y gráfico integrados y funcionando en conjunto, se construirá el entorno interactivo utilizando GDScript donde se añadirá una interfaz gráfica que permita al usuario modificar parámetros clave (como los factores ambientales) e interactuar con la simulación en tiempo real.

- **Pruebas de estrés y evaluación de resultados (2 semanas):** Se llevarán a cabo simulaciones en distintos escenarios de carga para evaluar la estabilidad de la herramienta. Durante esta fase se monitorizará el uso de la memoria gráfica, la carga computacional de la GPU y la tasa de fotogramas, realizando si es necesario optimizaciones finales para garantizar la estabilidad del sistema.
- **Redacción de la memoria (4 semanas):** Aunque la documentación del código y la recopilación de referencias se realizará desde el inicio a la vez que el desarrollo del sistema, será en las últimas semanas cuando se llevará a cabo la redacción formal de esta memoria, estructurando la metodología, analizando los resultados obtenidos y elaborando las conclusiones finales del proyecto.

1.3.1. Diagrama de Gantt



Capítulo 2

Estado del Arte y Marco Teórico

2.1. Fundamentos de la Informática Gráfica

La evolución de la informática gráfica se ha caracterizado históricamente por intentar alcanzar un mayor realismo visual, exactitud geométrica y rigor físico, lo que ha conllevado el desarrollo de estructuras de datos cada vez más complejas y modelos matemáticos capaces de aprovechar al completo las arquitecturas de hardware de cada época.

En sus inicios, la solución estándar para renderizar los objetos tridimensionales consistió en modelar únicamente la superficie externa, una forma eficiente con un bajo coste computacional pero que presentaba problemas para ciertas simulaciones físicas ya que realmente eran objetos huecos que carecían de densidad.

Durante las primeras décadas la representación de la estructura 3D de los objetos se llevaba a cabo mediante vértices, que son puntos en el espacio definidos por coordenadas cartesianas (x, y, z) y aristas que los unían. La unión de tres de estos vértices forma un triángulo, lo que se conoce como cara, que es la unidad fundamental con la que está constituida una malla poligonal (*Boundary Representation* o B-Rep) debido a su eficiencia para la tecnología de la época [1].

Una vez construida la malla se requiere de un proceso para su visualización conocido como rasterización que consiste en aplicar cálculos espaciales para adaptar los objetos 3D a la perspectiva de la cámara y proyectarlos de forma plana sobre las coordenadas de la pantalla. Una vez que los triángulos están proyectados en el plano 2D el sistema evalúa matemáticamente el área de los triángulos proyectados para identificar qué píxeles concretos quedan

cubiertos por ellos. Finalmente, para los píxeles de cada triángulo se le aplica la interacción lumínica siguiendo modelos como el sombreado de Phong o Gouraud, que calculan la interacción de la luz estimando la reflexión difusa y especular basándose en la normal de la malla.

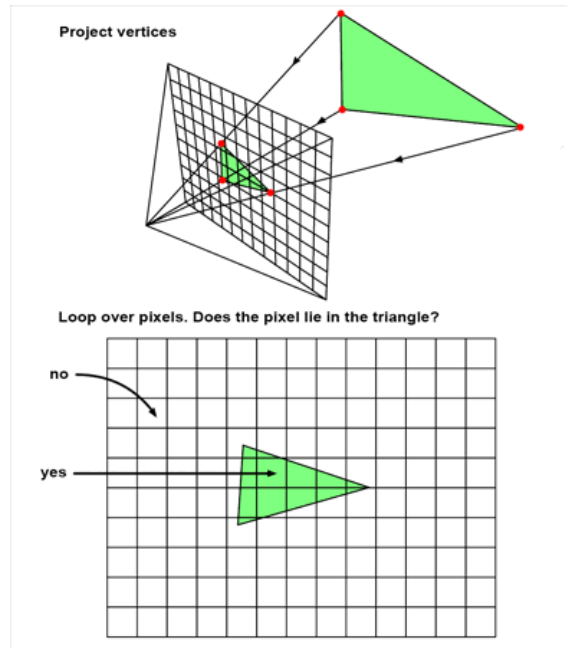


Figura 2.1: Proceso de Rasterización

Todo este proceso se lleva a cabo en la unidad de Procesamiento Gráfico (GPU) la cual a diferencia de una unidad central (CPU), diseñada para ejecutar instrucciones secuenciales, la GPU posee una arquitectura masivamente paralela ideal para calcular el color de millones de píxeles simultáneamente.

Para implementar físicamente este proceso de renderizado, la GPU utiliza un flujo de trabajo estándar conocido como *Pipeline* Gráfico (Cauce Gráfico) que describe la secuencia de operaciones físicas y lógicas por las que atraviesan los datos desde su concepción matemática hasta convertirse en la imagen proyectada al usuario.

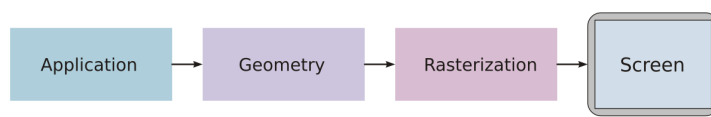


Figura 2.2: Esquema macroestructural del Pipeline Gráfico tradicional

Este proceso unidireccional comienza con la etapa de **Aplicación** en la CPU, que empaqueta y envía los datos de la memoria principal a la memoria de la tarjeta gráfica pasando a la fase de **Geometría**, donde programas especializados de la GPU (*Shaders*) trasladan los modelos al espacio bidimensional de la cámara y aplican la perspectiva para descartar automáticamente la geometría que queda fuera del rango de visión. Finalmente, los datos entran en la fase de **Rasterización** explicada anteriormente calculando los píxeles y colores que se muestran en pantalla y enviando esta información al monitor correspondiente.

Aunque el *pipeline* de rasterización se consolidó como el estándar en tiempo real, el cálculo de la luz era solo una aproximación por lo que para lograr un fotorrealismo superior, se desarrolló el **Trazado de Rayos** (*Ray Tracing*). A diferencia del cauce de rasterización convencional que proyecta polígonos en un plano, el trazado de rayos simula el comportamiento físico de la luz lanzando rayos virtuales desde la cámara hacia la escena. Al calcular analíticamente las intersecciones exactas de los rayos contra la malla, logra representar reflejos y sombras de una forma mucho más exacta y realista que las aproximaciones geométricas de la rasterización [15].

Sin embargo, a pesar de la enorme eficiencia en el cálculo y visualización de mallas ambos métodos comparten una gran limitación estructural: solo modelan la frontera exterior de los objetos. Esta restricción las hace computacionalmente inviables para simular fenómenos sin fronteras definidas, como el fuego o el humo.

Los primeros intentos de simular dichos fenómenos se realizaron superponiendo capas de polígonos semitransparentes (*alpha-blending*) pero esto fracasó debido a que generaban errores visuales al calcular qué capa estaba delante de otra (*depth-sorting*) y no lograban simular cómo la luz iluminaba el interior del fluido. Para solucionar este problema, pioneros como James Kajiya y Brian Von Herzen demostraron en 1984, con su trabajo fundacional *Ray Tracing Volume Densities*[12], que era necesario dejar de modelar solo la superficie exterior de los elementos y pasar a un enfoque de modelado volumétrico. Consolidado teóricamente posteriormente por Nelson Max en 1995[16], el cual transformó conceptualmente el espacio de un simple contenedor pasivo a una matriz tridimensional activa, capaz de almacenar datos continuos de densidad y temperatura en cada punto.

Para que el hardware pudiera procesar esta matriz activa, fue necesario dar un salto evolutivo en la gestión de la memoria gráfica. Mientras que los modelos superficiales clásicos detallan su geometría ^{en}volviéndola con imágenes planas mediante coordenadas bidimensionales (U, V), la representación de fenómenos sin fronteras exige el uso de texturas tridimensionales (cono-

cidas como *Volume Textures*).

A nivel interno, una textura 3D se organiza como una densa cuadrícula cúbica de *texels* que son la unidad mínima de información de una textura, siendo el equivalente a un píxel pero dentro del espacio de la memoria gráfica. Para extraer información de su interior, el sistema gráfico utiliza coordenadas espaciales (U, V, W) normalizadas en el rango continuo de 0.0 a 1.0. Esta normalización es clave, ya que permite activar el hardware de filtrado nativo de la GPU logrando que si un cálculo necesita conocer el estado de un punto situado justo entre varios vóxeles adyacentes, la tarjeta gráfica resuelve automáticamente una interpolación trilineal. El resultado es una lectura de datos (como la temperatura o la biomasa) sin transiciones abruptas ni artefactos visuales.

En definitiva, la adopción de estas texturas permitió guardar y consultar de forma enormemente eficiente las propiedades físicas de un entorno, estableciendo así las bases para los algoritmos de renderizado volumétrico modernos.

2.2. Renderizado Volumétrico

Para dar una solución visual a las matrices tridimensionales continuas, la informática gráfica tuvo que cambiar drásticamente las técnicas que usaba para el renderizado pasándose a usar métodos de renderizado de volúmenes.

Por una parte tenemos el renderizado de volúmenes indirecto (IVR) que intentan extraer una malla poligonal a partir de los datos espaciales, destacando el algoritmo de *Marching Cubes*. El funcionamiento de este algoritmo se basa en analizar el volumen dividiéndolo en cubos lógicos formados por ocho vóxeles vecinos comparando para cada cubo el valor escalar de sus ocho vértices $(v_1, v_2, \dots, v_8)$ frente a un umbral o iso-valor predefinido (I) [17].

Si el valor del vértice comparado es mayor o igual al umbral ($v_i \geq I$), se clasifica como dentro de la superficie (1) mientras que si es menor queda fuera (0), generando así con los 8 vértices una combinación de ceros y unos que se consulta en una tabla precalculada (*look-up table*) en la que se indica exactamente qué configuración de triángulos debe dibujarse dentro de ese cubo.

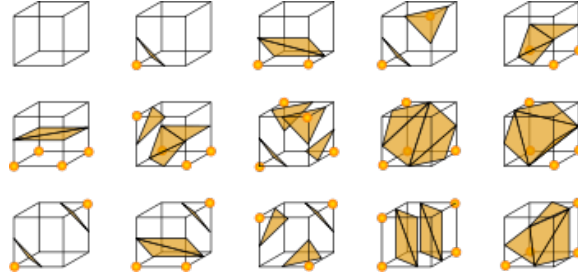


Figura 2.3: Posibles casos de MarchingCubes

Sin embargo, esta técnica presenta dos limitaciones críticas.

- Su complejidad algorítmica hace que necesite realizar una enorme cantidad de cálculos computacionales para generar mallas de alta resolución.
- Su función principal es construir una frontera artificial que envuelve los datos, lo que le impide visualizar las estructuras internas y las continuas variaciones de densidad.

Por este motivo, para el renderizado de fenómenos como el humo o el agua se optó por el Renderizado Volumétrico Directo (DVR) que al contrario que el IVD sí permite visualizar el interior de los volúmenes y no necesita generar ninguna geometría intermedia ahorrándole esa carga computacional. Para lograr esto, se desarrolló el Trazado de Rayos Volumétrico (*Volumetric Ray Tracing*), una técnica donde los rayos generados desde la cámara ya no buscan colisionar contra una superficie externa, sino que atraviesan directamente todo el volumen de datos, calculando la absorción y dispersión de la luz en su interior[9].

La implementación algorítmica y optimizada de esta técnica en tiempo real se conoce como *Raymarching*. Es un algoritmo que en lugar de procesar todo el volumen de golpe, lanza un rayo desde la cámara hacia el volumen el cual cada cierta distancia va tomando muestras de los valores del voxel en el que se encuentre, como densidad o temperatura, para ir sumándolos y calcular el color final del píxel hasta que el rayo abandona el volumen o se vuelve completamente opaco[16].

Para calcular el color final resultante para un píxel se resuelve mediante la siguiente ecuación discreta:

$$C_{total} = \sum_{i=0}^N C_i \alpha_i \prod_{j=0}^{i-1} (1 - \alpha_j)$$

Donde las variables son:

- C_{total} : Es el color final y opacidad resultante que se mostrará en el píxel de la pantalla.
- $C_i\alpha_i$: Representa la contribución de color radiante (C_i) ponderada por su propia opacidad o densidad local (α_i) en el paso exacto i en el que se encuentra el rayo.
- $\prod_{j=0}^{i-1}(1 - \alpha_j)$: Es el porcentaje de luz que logró sobrevivir y llegar hasta el paso i . Lo hace multiplicando la transparencia inversa ($1 - \alpha$) de todos los pasos anteriores (j) que el rayo ya ha atravesado.

Hasta este punto, técnicas como el DVR y el *Raymarching* funcionan sin problema para la visualización de una matriz tridimensional estática pero para simular fluidos naturales y fenómenos como el humo o el fuego es necesario dotar a ese volumen de movilidad y dinámicas físicas en tiempo real.

Para lograr esta movilidad en tiempo real, se tuvo que trasladar la carga computacional a la tarjeta gráfica debido al alto número de cálculos y actualizaciones que se necesitan, mapeando la cuadrícula espacial del renderizado sobre texturas tridimensionales alojadas en la memoria de vídeo (VRAM)[14].

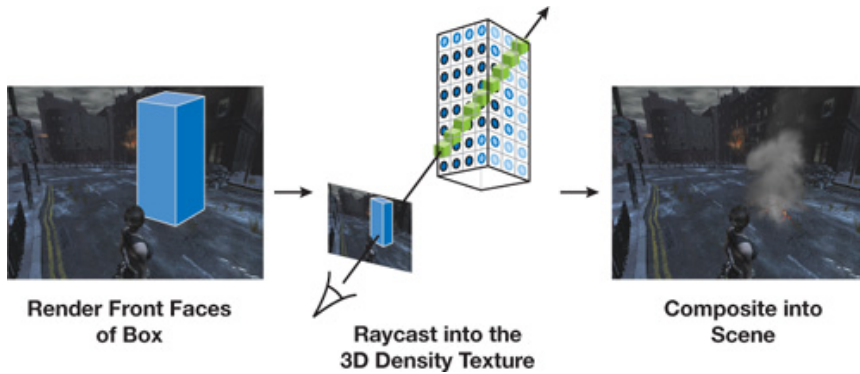


Figura 2.4: Descripción conceptual del trazado de rayos

Aprovechando la arquitectura de la GPU, en lugar de calcular trayectorias de partículas individuales, se utiliza programas con una alta capacidad de paralelizar cálculos, como los *Compute* o *Fragment Shaders*, y así evaluar simultáneamente millones de celdas estáticas. Dado que la GPU no permite leer y escribir sobre el mismo bloque de memoria de forma segura al mismo tiempo se mantienen dos texturas 3D idénticas donde en cada iteración el *shader* lee los datos de una y escribe el resultado en la segunda, cambiando los

roles en el siguiente ciclo. Esta técnica es conocida como *Ping-Ponging*[13].

Una vez que la infraestructura de memoria está lista para la actualización dinámica y paralela, se necesitaba definir un marco matemático que gobernase su evolución temporal. Para ello se recurre a la investigación operativa a través de los **Autómatas Celulares**[18], los cuales se adaptan perfectamente al funcionamiento del *Raymarching* y permiten evaluar cada celda espacial de forma iterativa mediante reglas de transición basadas en su situación actual y en las condiciones de sus vóxeles adyacentes.

Finalmente al juntar estas dos técnicas se consigue transformar un contenedor volumétrico estático en un ecosistema reactivo idóneo para la creación de simuladores en tiempo real como los de incendios forestales.

2.3. Modelos Físicos de Propagación de Incendios Forestales

El estudio de la propagación y el avance de los incendios forestales se ha fundamentado históricamente en la necesidad de predecir el comportamiento del fuego para la gestión de emergencias.

Las primeras aproximaciones analíticas, que siguen siendo el estándar en la actualidad, se basaron en modelos matemáticos empíricos destacando el modelo cuasi-estacionario de Rothermel (1972), el cual calcula la tasa de propagación frontal (R) del fuego basándose en la carga física del combustible, la topografía y la meteorología[4]:

$$R = \frac{I_R \xi}{\rho_b \epsilon Q_{ig}}$$

Esta formulación divide la velocidad de propagación entre las fuerzas de empuje térmico y los sumideros de energía, que representan la resistencia de las plantas al tener que absorber calor para evaporar su humedad antes de prenderse. Sus variables principales son:

- I_R (**Intensidad de reacción**): Energía bruta liberada por unidad de área activa.
- ξ (**Flujo de calor propagado**): Fracción de energía convectiva/radiante proyectada hacia la vegetación frontal intacta.
- ρ_b (**Densidad aparente**): Masa de la biomasa por volumen.
- ϵ (**Calentamiento efectivo**): Eficiencia térmica de absorción.

- Q_{ig} (**Calor de pre-ignición**): Energía necesaria para evaporar la humedad y alcanzar la pirólisis.

Posteriormente, en la década de 1980 y principios de 1990, se estandarizó como gran alternativa internacional el Sistema Canadiense de Predicción del Comportamiento del Fuego (FBP) que, a diferencia del balance energético de Rothermel, es estrictamente empírico. Se apoya en bases de datos masivas obtenidas de quemas controladas y de la observación de incendios forestales reales en bosques boreales, usando estos datos para relacionar directamente la velocidad de propagación empírica con el tipo de combustible topográfico y el Índice Meteorológico de Peligro de Incendio (FWI, *Fire Weather Index*) a través de ecuaciones de regresión, ofreciendo resultados muy precisos en ecosistemas concretos[6][7].

La ecuación principal para determinar la tasa de propagación base según el modelo FBP se define como:

$$ROS = a \cdot (1 - e^{-b \cdot ISI})^c$$

En esta formulación, a diferencia del modelo de Rothermel los componentes son puramente estadísticos y meteorológicos:

- a, b, c (**Constantes de regresión**): Son coeficientes empíricos precalculados para cada uno de los 16 modelos de combustible estándar definidos en Canadá (como coníferas, frondosas o pastos).
- ISI (**Initial Spread Index**): Es un valor numérico del sistema FWI que combina la velocidad del viento con el contenido de humedad de los combustibles finos y que representa el potencial de propagación inmediato del fuego sin tener en cuenta la acumulación de combustible pesado.

Finalmente, esta propagación base se ajusta multiplicándola por un factor de efecto de acumulación, que introduce la influencia del combustible forestal de mayor grosor. Al basarse en coeficientes de regresión es computacionalmente mucho más ligero que Rothermel, por lo cual sistemas, como el simulador nacional canadiense *Prometheus*, pueden calcular grandes expansiones de forma muy ágil, aunque perdiendo la capacidad de usarse en ecosistemas para los que no existan grandes cantidades de datos estadísticos previos.

Tanto las ecuaciones de Rothermel como las del FBP calculan la propagación en un punto específico de forma unidimensional por lo que la simulación espacial determinista de los años 90 adoptó técnicas de propagación vectorial basadas en el **Principio de Huygens**. Este principio, originario de la física ondulatoria, establece que cada punto de un frente de onda actúa

como un nuevo emisor de ondas secundarias, siendo la suma de todos los bordes de estas ondas secundarias la que forman la siguiente onda grande[5].

Al adaptar este concepto matemático a los incendios forestales, el frente de onda es la línea de fuego donde se asume que cada punto activo se comporta como un foco de ignición independiente que genera su propia área de propagación. La envolvente exterior de todas estas áreas conforma el perímetro actualizado del incendio, permitiendo simular con gran precisión expansiones asimétricas, flancos que avanzan a distinta velocidad y cambios abruptos en la dirección del viento sin que el frente del fuego se rompa matemáticamente.

A partir de la fusión de estos modelos de propagación han surgido distintos simuladores que dominan la gestión de emergencias a nivel mundial. Por un lado, sistemas como *FARSITE* y *BehavePlus*, desarrollados por el Servicio Forestal de EE. UU., utilizan el modelo matemático físico de Rothermel combinado con la propagación elíptica de Huygens. Por otro lado, *Prometheus*, el simulador nacional de Canadá, emplea el mismo motor de propagación vectorial de Huygens pero alimentado matemáticamente por las ecuaciones empíricas de su propio sistema nacional (FBP).

En el panorama nacional español resalta la labor del Laboratorio de Defensa contra Incendios Forestales de la Universidad de Córdoba (LABIF-UCO), el cual bajo el liderazgo del Dr. Francisco Rodríguez y Silva, ha desarrollado sistemas como UCO40, que cataloga 40 modelos de combustible superficial para afinar el comportamiento térmico de Rothermel en formaciones endémicas andaluzas[19].

De forma paralela, otros enfoques más modernos han optado por la Dinámica de Fluidos Computacional (CFD) para resolver las complejas ecuaciones espaciales de Navier-Stokes aplicadas a la combustión y la convección térmica. Sin embargo, tanto los modelos de expansión vectorial como los sistemas CFD tienen una gran limitación práctica: su altísimo coste computacional hace imposible una simulación a gran escala en tiempo real, ofreciendo además salidas puramente analíticas (mapas isócronos de contornos 2D) sin inmersión visual para los operativos.[20]

2.3.1. Autómatas Celulares

Como se ha expuesto, aunque los enfoques vectoriales y los modelos continuos de fluidos (CFD) ofrecen un notable rigor físico, su naturaleza matemática continua y su dependencia de cálculos globales (como la intersección de polígonos o la resolución de pesadas ecuaciones de fluidos) generan

un gran cuello de botella computacional que les impide llevar a cabo su ejecución en tiempo real sobre grandes extensiones de terreno.

Para superar esta barrera arquitectónica, la investigación se vió obligada a abandonar esos modelos continuos y evolucionar hacia modelos de simulación basados en cuadrículas discretas mediante el uso de Autómatas Celulares (CA).

Un autómata celular es un modelo matemático compuesto por una matriz regular de celdas donde cada una posee un estado perteneciente a un conjunto finito de posibilidades (en el contexto forestal, típicamente: intacto, en ignición, ardiendo o calcinado). El funcionamiento se basa en la evolución iterativa de todo el sistema, para cada iteración temporal, el nuevo estado de una celda se calcula aplicando un conjunto definido de reglas de transición que evalúan tanto sus condiciones internas como el estado de las celdas adyacentes que conforman su vecindad (normalmente las ocho celdas que la rodean en el caso de un plano bidimensional, lo que se conoce como vecindad de Moore)[18].

Esta arquitectura se puede trasladar a fenómenos complejos como la propagación de un gran frente de llamas a partir de interacciones locales al depender únicamente de su entorno más inmediato. Esto logra que la actualización del estado de cada celda sea totalmente independiente del resto del mapa, lo que convierte a los autómatas celulares en estructuras ideales para ser paralelizadas masivamente en sistemas de cómputo modernos.

El enfoque contemporáneo más aceptado es el modelo estocástico propuesto por Alexandridis[21], cuya precisión quedó demostrada al simular a posteriori incendios devastadores como el de Spetses (1990) y Rodas (2008), prediciendo áreas quemadas con un margen de error mínimo y sin reajustes artificiales.

En este modelo, la probabilidad de ignición (P_{ign}) temporal de una celda sana vecina a una celda en llamas se define como:

$$P_{ign} = P_0 \cdot (1 + p_w) \cdot (1 + p_s) \cdot p_h$$

Esta ecuación desglosa la termodinámica de Rothermel en multiplicadores independientes que pueden procesarse en paralelo para millones de celdas de forma instantánea:

- P_0 (**Probabilidad Base**): Representa la inflamabilidad inherente al tipo de combustible forestal.
- p_w (**Factor de Viento**): Se calcula exponencialmente, penalizando o bonificando la propagación según si el viento sopla a favor o en contra

de la celda vecina:

$$p_w = \exp(c_1 \cdot V \cdot (\cos(\theta) - 1))$$

Donde V es la velocidad del viento, θ es el ángulo entre la dirección del viento y la línea de propagación, y c_1 es la constante empírica de ponderación del viento.

- p_s (**Factor de Pendiente**): Refleja cómo la convección térmica precalienta el combustible de forma más eficiente cuesta arriba:

$$p_s = \exp(a \cdot \tan(\theta_s))$$

Donde θ_s es el ángulo de inclinación del terreno local y a es la constante empírica de pendiente.

- p_h (**Factor de Humedad**): Se encarga de frenar el avance del fuego, ya que el agua contenida en la vegetación debe evaporarse antes de alcanzar la pirólisis. Se modela como un decaimiento exponencial:

$$p_h = \exp(-c_2 \cdot H)$$

Donde H es el índice de humedad relativa (de 0 a 1) y c_2 es la constante de resistencia térmica a la humedad.

Parámetro CA de Alexandridis	Valor Típico	Definición
Constante de Viento (c_1)	0.045	Ponderación del flujo convectivo.
Constante de Humedad (c_2)	0.131	Resistencia térmica a la humedad.
Constante de Pendiente (a)	0.078	Constante empírica de la pendiente.
Probabilidad Base (P_0)	0.580	Probabilidad base bajo condiciones de referencia mediterráneas.

Tabla 2.1: Valores empíricos de calibración para el Autómata Celular basados en el incendio de Spetses.

2.3.2. Método de Monte Carlo

A pesar de los avances en los modelos empíricos y los Autómatas Celulares, la predicción de incendios forestales se enfrenta a otro problema fundamental. Variables importantes como la velocidad y dirección del viento o el grado de humedad del combustible rara vez se conocen con precisión y en muchas de las ocasiones sufren cambios repentinos que pueden conllevar a un comportamiento del fuego totalmente distinto al calculado inicialmente.

Para resolver esta vulnerabilidad se pasó de usar un enfoque determinista, donde se produce un único resultado que ignora la variabilidad del entorno, a predicciones probabilísticas utilizando el Método de Montecarlo. Este método consiste en ejecutar la simulación de propagación del fuego un número elevado de veces (N iteraciones), introduciendo en cada una de ellas pequeñas variaciones aleatorias en los parámetros de entrada dentro de un rango de varianza estadística controlada[22].

Al agrupar los resultados, se obtiene un mapa de riesgo donde la probabilidad de que una celda espacial sea alcanzada por el fuego se define como la razón entre el número de simulaciones en las que dicha celda ardió y el total de iteraciones[23]:

$$P_{quemado}(x, y, z) = \frac{1}{N} \sum_{i=1}^N S_i(x, y, z)$$

Donde N es el número total de simulaciones del conjunto y $S_i(x, y, z)$ indica el estado final de la celda en la iteración i (siendo 1 si fue quemada y 0 si permaneció intacta).

Hoy en día, la potencia de las GPU permite ejecutar estas predicciones en tiempo real lo que permite que dos grandes beneficios:

- **Evaluación de riesgos a largo plazo:** Ejecutando miles de igniciones aleatorias sobre grandes escalas de paisaje para identificar áreas críticas o cuellos de botella topográficos, generando lo que se conoce como Mapas de Probabilidad de Quema (*Burn Probability Maps*).
- **Predicción operativa en tiempo real :** Durante una emergencia activa puede simular múltiples escenarios futuros de un mismo incendio para reasignar recursos de una manera eficiente minimizando pérdidas.

No obstante, aunque estos sistemas mejoraron la velocidad de cálculo, la visualización seguía estando limitada en gran medida a entornos bidimensionales o representaciones topográficas esquemáticas.

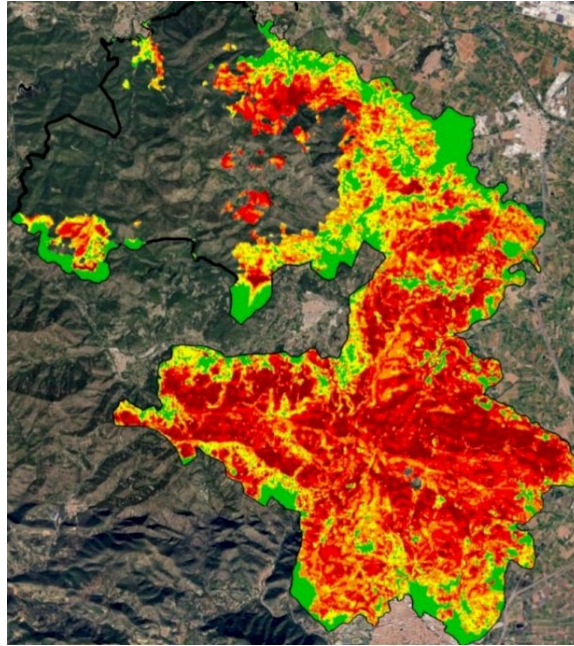


Figura 2.5: Ejemplo de mapa de calor 2D de un incendio

A pesar de los grandes avances tanto en el renderizado 3D como en el cálculo de las probabilidades de incendios todavía nos encontramos con una gran desconexión entre ellos. Por un lado, los simuladores forestales científicos, nutridos con rigor termodinámico, carecen de representaciones volumétricas inmersivas que dificultan la comprensión del volumen y la orografía real, mientras que por el otro, la industria gráfica y los motores de videojuegos disponen de una avanzada tecnología de renderizado volumétrico en tiempo real pero que generan sus fluidos alimentando las ecuaciones ópticas mediante algoritmos de ruido procedimental.

Este proyecto propone fusionar ambos paradigmas, desarrollando un simulador interactivo en el motor Godot 4 que delegue la lógica matemática de propagación basada en Autómatas Celulares a la GPU mediante el uso de *Compute Shaders*. Además de usar Modelos Digitales de Elevación reales (*Heightmaps*) como entorno físico y *Raymarching* como técnica de renderización volumétrica, consiguiendo así una herramienta con un rigor matemático de la prevención de incendios y fidelidad visual.

Capítulo 3

Metodología

En este capítulo se detalla el proceso de diseño e implementación del simulador volumétrico de incendios forestales. A lo largo del mismo, se justificará la elección del entorno de desarrollo, se desglosará la arquitectura del autómata celular y la implementación de los shaders necesarios, se explicará la integración de datos topográficos reales para la generación del terreno y, finalmente, se comentarán los mecanismos de interacción desarrollados y los principales retos arquitectónicos encontrados.

3.1. Selección de herramientas

La selección del entorno de desarrollo es una decisión importante que afecta al rendimiento y la escalabilidad del proyecto. A diferencia de otros software, un simulador de incendios forestales en tiempo real basado en autómatas celulares necesita procesar una gran cantidad de datos topográficos y realizar numerosos cálculos para su correcto funcionamiento.

Es por esto que la decisión no se ha basado en las capacidades de renderizado fotorrealista del motor, sino en los siguientes requisitos técnicos:

- **Capacidad de cómputo en paralelo.** Para simular la propagación del fuego a través de la matriz volumétrica es necesario que tenga soporte para la ejecución de *Compute Shaders* permitiendo actualizar el estado de miles de celdas en tiempo real sin saturarse.
- **Control sobre la memoria VRAM.** Es necesario disponer de un gran control de la memoria de video para gestionar los buffers de datos y la sincronización para la comunicación entre la GPU y la CPU sin ocasionar cuellos de botella.
- **Gestión dinámica de datos geoespaciales.** Para el funcionamiento, los datos que se introducirán son Modelos Digitales de Elevaciones

(DEM) y mapas de clasificación de vegetación, por lo que es necesario que se proporcionen interfaces que faciliten su manipulación y conversión a estructuras 3D.

- **Curva de aprendizaje y ecosistema desacoplado.** El entorno debe proporcionar una interfaz clara con una buena documentación, además de una estructura modular que permita separar de forma lógica las distintas interfaces de las que dispondrá el simulador.

3.1.1. Comparativa de motores gráficos

Para el desarrollo de este simulador, la elección del motor gráfico se ha basado en los requisitos técnicos mencionados anteriormente, evaluando los tres motores de desarrollo con mayor cuota de mercado y soporte de la industria:

- **Unreal Engine:** Es el motor líder en fidelidad gráfica y rendimiento ofreciendo un control de bajo nivel mediante C++. Sin embargo, para la implementación de *Compute Shaders* necesita una gran cantidad de código de configuración ya que su arquitectura está fuertemente acoplada a su propio *pipeline* de renderizado. En cuanto a portabilidad, aunque es el estándar para consolas de última generación y entornos Windows, la compilación cruzada hacia Linux o macOS suele necesitar configuraciones del entorno muy pesadas, mientras que para plataformas móviles genera aplicaciones con una huella de memoria enorme además de haber descontinuado oficialmente su soporte nativo para exportación web (HTML5).
- **Unity:** Es una de las herramientas más versátiles del mercado y cuenta con soporte nativo para la ejecución de *Compute Shaders* en HLSL. Pero tiene ciertas limitaciones para esta simulación, al depender de un recolector de basura (*Garbage Collector*), que provoca picos de latencia al transferir grandes matrices de datos entre la CPU y la GPU. A nivel de portabilidad, Unity destaca históricamente por su robustez para exportar proyectos para los sistemas operativos más usados y sistemas móviles, no obstante aunque permite compilar para navegadores web mediante WebGL, genera *builds* con un gran tamaño base que repercuten negativamente en los tiempos de carga en entornos no especializados.
- **Godot:** Es un motor de código abierto con licencia MIT. A partir de su versión 4.X ha integrado la API Vulkan la cual resulta muy determinante ya que contiene la interfaz **RenderingDevice**, que permite un acceso directo de bajo nivel a la GPU. Esto permite compilar y ejecutar *Compute Shaders* (escritos en GLSL puro) con un control absoluto

sobre la memoria VRAM y los *buffers* de datos. A nivel de portabilidad, Godot destaca por su excepcional agilidad: su propio editor permite compilar ejecutables nativos para Windows, Linux, macOS, Android e iOS en cuestión de segundos y sin requerir pesados compiladores externos ni dependencias adicionales. También es relevante su optimizado soporte para la plataforma web (HTML5/WebAssembly), generando binarios extremadamente ligeros.

Característica	Godot 4	Unity	Unreal Engine 5
Soporte <i>Shaders</i>	Óptimo (Nativo GLSL)	Bueno (Nativo HLSL)	Complejo (Abundante configuración C++)
Acceso VRAM	Directo (<i>RenderingDevice</i>)	Intermediado (<i>ComputeBuffers</i>)	Abstraído (<i>Render Graph</i>)
Portabilidad (S.O.)	Ágil e integrada (Compilación cruzada directa sin dependencias externas)	Buena (Requiere pesados módulos anclados al sistema)	Enfocada a consolas/Windows, herramientas de compilación pesadas
Exportación Web	Excelente (Binarios ligeros y carga rápida en HTML5/WASM)	Funcional (<i>Runtime</i> muy pesado, largos tiempos de carga)	Descontinuada
Lenguajes de desarrollo	GScript, C++, C#, GLSL	C#, HLSL	C++, Blueprints, HLSL
Arquitectura de software	Sistema de Nodos (Orientado a la composición jerárquica)	Entidad-Componente (<i>MonoBehaviours</i> / ECS)	<i>Actors</i> y Componentes (Acoplado al núcleo central)
Huella del entorno	Ultra ligero (Unos 100 MB, editor autónomo y portable)	Pesado (Varios GBs, dependiente del <i>Unity Hub</i>)	Muy pesado (Mas de 30 GBs de base estructural)
Sobrecarga (<i>Overhead</i>)	Muy baja (Estable en tiempo real)	Media (Picos de latencia por <i>G. Collector</i>)	Muy alta (Decenas de servicios en segundo plano)
Licencia	MIT (Libre y código abierto)	Privada (Suscripción o tarifa por ingresos)	Privada (Regalías por ingresos brutos)

Tabla 3.1: Comparativa técnica de los motores gráficos evaluados frente a los requisitos arquitectónicos y de despliegue del simulador.

Llevando a cabo un análisis de todas estas características de los diferentes entornos de desarrollo y teniendo en cuenta que el núcleo del proyecto implica una comunicación constante, masiva y de bajísima latencia con la GPU, se llega a la conclusión de que Godot es la mejor opción. Dicho entorno resuelve esta necesidad a través de la API `RenderingDevice` de Vulkan, ofreciendo un control de memoria que esquivaba la sobrecarga estructural de Unreal y la recolección de basura de Unity. Además, a esto se le suma un *overhead* mínimo y una portabilidad que, comparada con los otros motores, resulta excepcional permitiendo compilar binarios ultraligeros y de despliegue directo tanto en sistemas de escritorio como en entornos web. Todo ello bajo una licencia de código abierto (MIT) ideal para la investigación científica, que consolida finalmente a Godot como la infraestructura tecnológica más eficiente, versátil y racional para afrontar los retos técnicos de este simulador.

3.2. Arquitectura del Simulador

El diseño de la arquitectura de este proyecto se ha dirigido con la necesidad de tratar de manera eficiente una simulación con un gran volumen de datos. Para ello, se ha establecido un modelo de procesamiento híbrido donde la CPU se encarga de la lógica de control y la administración del flujo de la aplicación mientras que la GPU actúa como motor físico para ejecutar el cálculo masivo en paralelo. A lo largo de esta sección se explicará cada uno de los componentes de esta arquitectura, desde los fundamentos tecnológicos para su correcta comprensión hasta la implementación técnica en la GPU y sincronización de memorias.

3.2.1. Fundamentos tecnológicos: Arquitectura de la GPU y el *Pipeline* Gráfico Tradicional

Para comprender la base computacional de este simulador, es necesario entender primero la diferencia arquitectónica fundamental entre un procesador central (CPU) y una tarjeta gráfica (GPU). La CPU está diseñada bajo un paradigma de baja latencia, por lo que posee un número reducido de núcleos muy complejos y veloces, optimizados para ejecutar instrucciones secuenciales, saltos lógicos condicionales y administrar el sistema operativo.

Por el contrario, la GPU está diseñada para lograr un alto rendimiento, lo que conlleva a su arquitectura a usar un mayor número de núcleos pero siendo la complejidad de cada uno de ellos mucho menor. Una gráfica moderna está compuesta por miles de pequeñas Unidades Aritmético-Lógicas (ALUs) diseñadas para ejecutar la misma instrucción matemática sobre múltiples datos de forma simultánea. Esta estructura le proporciona un paralelismo que es ideal para procesar gráficos ya que calcular el color de un

píxel en la pantalla es independiente del cálculo del píxel adyacente[1].

Para alimentar de datos a estos miles de núcleos simultáneamente sin generar cuellos de botella, la GPU cuenta con su propia memoria, la **VRAM** (*Video Random Access Memory*). Esta memoria, a diferencia de la memoria RAM administrada por la CPU, posee un ancho de banda masivo diseñado específicamente para operaciones de lectura y escritura altamente paralelas.

En el contexto del renderizado visual tradicional la VRAM se organiza en buffers que deben estar estrictamente formateados en estructuras muy específicas, destacando[24][8]:

- **Vertex Buffers (VBO):** Almacenan los atributos matemáticos de los vértices que componen una malla 3D, No solo guardan las coordenadas espaciales, sino también datos acoplados a cada vértice, como los vectores normales.
- **Index Buffers (EBO/IBO):** Este buffer almacena únicamente una lista de índices enteros que indica a la GPU en qué orden exacto debe conectar los vértices previamente cargados para generar la malla 3D.
- **Buffers de Texturas:** Son estructuras de memoria bidimensionales (y en ocasiones tridimensionales) altamente optimizadas por hardware para el muestreo espacial. Normalmente solo contienen los mapas de color (albedo), rugosidad o desplazamiento aplicados sobre la geometría.

Históricamente, para aprovechar la potencia de los núcleos y procesar estos *buffers*, los datos debían atravesar un flujo de trabajo unidireccional el cual consta de varias etapas secuenciales para convertir los datos tridimensionales en una imagen 2D:

1. **Ensamblaje de entrada (*Input Assembler*):** La GPU recibe desde la CPU los datos crudos de los modelos 3D (vértices, normales, coordenadas UV).
2. **Vertex Shader:** Un programa ejecutable que transforma las coordenadas 3D locales de los vértices a las coordenadas 2D de la pantalla de la cámara.
3. **Rasterización:** El hardware matemático toma los triángulos formados por los vértices y determina qué píxeles de la pantalla (fragmentos) están cubiertos por dicha geometría.
4. **Fragment Shader:** Un segundo programa que calcula el color final, la iluminación y las sombras para cada uno de esos fragmentos.

5. **Fusión de salida (*Output Merger*):** Se combinan los fragmentos procesados y se envían al monitor para mostrar la imagen final.

3.2.2. Fundamentos tecnológicos: El *Pipeline* de Cómputo

Antes de abordar la solución computacional, es preciso definir formalmente qué es un *shader*, y no es más que un programa diseñado específicamente para ser compilado y ejecutado directamente en los núcleos de la GPU. En motores como Godot, estos *shaders* tradicionales se construyen usando un lenguaje de alto nivel propio (*Godot Shading Language*), el cual abstrae gran parte de la configuración de bajo nivel y está optimizado para integrarse de manera fluida con el motor de renderizado[25].

Sin embargo, para el desarrollo de un simulador volumétrico basado en autómatas celulares se necesita actualizar el estado de miles de celdas en cada iteración. Hacer esto de forma secuencial en la CPU generaría un cuello de botella inmenso lo que lo hace inviable para una simulación en tiempo real, y utilizar los *shaders* visuales de Godot obligaría a empaquetar datos matemáticos como si fueran colores.

Este problema se ha resuelto haciendo uso del paradigma GPGPU (*General-Purpose computing on Graphics Processing Units*), que consiste en dejar de usar la tarjeta gráfica de forma exclusiva para la visualización y pasar a usarla como un coprocesador matemático masivamente paralelo aplicable a cualquier ámbito. En la práctica, este paradigma permite emplear la enorme cantidad de unidades aritmético-lógicas (ALUs) de la GPU para resolver operaciones matemáticas de propósito general de forma paralela, desligándose por completo del renderizado de imágenes y de las restricciones del *Pipeline* Gráfico tradicional[14].

Esto se ha hecho posible gracias a dejar de usar los *shader* tradicionales (Vertex y Fragment) para pasar a usar ***Compute Shaders*** los cuales, a diferencia de los tradicionales, operan de forma independiente al *pipeline* gráfico. Esta independencia les permite usar y estructurar los distintos *buffers* de memoria que necesiten haciéndolos ideales para inyectar los datos topográficos del terreno, calcular la propagación del fuego sobre la matriz del autómata celular en paralelo, y almacenar los resultados térmicos actualizados directamente en la VRAM sin necesidad de someter los datos a un proceso de dibujo en pantalla[13].

3.2.3. El motor de Simulación

Elección del modelo

La simulación computacional de la propagación de un incendio forestal es un problema matemático complejo que exige traducir procesos físicos y termodinámicos a algoritmos eficientes. En el ámbito de la ingeniería de software aplicada a la predicción de incendios, existen dos grandes enfoques arquitectónicos para modelar el crecimiento del frente de llamas: los modelos vectoriales y los modelos discretos.

Los **modelos vectoriales**, como ya se ha comentado con anterioridad, representan el avance del fuego como una serie de curvas geométricas que crecen a lo largo del tiempo siguiendo el Principio de Huygens[5]. Los ejemplos más destacados de este enfoque son simuladores profesionales como **FARSITE** (desarrollado por el Servicio Forestal de EE.UU.) o el sistema canadiense **Prometheus**.

- **Ventajas:** Ofrecen una alta precisión espacial y al trabajar con vectores, generan frentes de propagación suaves y visualmente muy realistas, evitando formas cuadriculadas.
- **Desventajas:** Su principal debilidad radica en la resolución topológica ya que cuando dos frentes de fuego colisionan, calcular la intersección matemática de sus polígonos para fusionarlos en una nueva curva requiere algoritmos de geometría secuenciales. Esta dependencia estructural los convierte en un enorme cuello de botella para la paralelización masiva en la GPU.

Por otro lado, los **modelos discretos** renuncian a la exactitud del espacio vectorial y dividen el entorno de simulación en una matriz finita (una cuadrícula 2D o un volumen 3D)[26]. Ejemplos históricos de este enfoque son las aproximaciones probabilísticas avanzadas como la adaptación estocástica de Alexandridis[21].

- **Ventajas:** Resuelven el problema de la colisión de dos frentes de fuego ya que simplemente múltiples celdas cambian su estado simultáneamente evaluando a sus vecinas, eliminando cualquier necesidad de calcular intersecciones vectoriales globales. Esta independencia espacial los hace altamente paralelizables, encajando a la perfección con la arquitectura de múltiples hilos de las tarjetas gráficas.
- **Desventajas:** Su precisión física está limitada estrictamente por la resolución de la matriz (el tamaño físico de cada voxel). Además, la propagación puede sufrir artefactos direccionales, tendiendo a generar patrones poco naturales debido a la limitación de movimiento impuesta por la vecindad de las celdas.

Debido a las características y objetivos de este proyecto, que requiere gestionar un volumen tridimensional y procesar los datos aprovechando la arquitectura paralela de la GPU, el uso de modelos vectoriales continuos resulta arquitectónicamente inviable y por lo tanto se toma la decisión de adoptar un modelo discreto mediante Autómatas Celulares.

Dentro del paradigma de los modelos discretos, la literatura científica presenta diferentes aproximaciones para resolver la propagación del fuego. Para la implementación de este motor lógico, se han evaluado las tres arquitecturas matriciales más representativas:

- **Modelos teóricos de percolación (ej. Drossel-Schwabl):** Son computacionalmente los más rápidos ya que operan bajo reglas binarias muy básicas en un bosque idealizado, pero carecen de variables termodinámicas reales (viento, pendiente o tipo de combustible), lo que los invalida para una simulación técnica realista[27].
- **Autómatas celulares deterministas (ej. Karafyllidis y Thanailakis):** Esta aproximación sí integra factores topográficos y meteorológicos, estableciendo ecuaciones con tiempos de retardo fijos. Su mayor defecto es que generan artefactos visuales muy severos (frentes de fuego con formas geométricas rígidas, como diamantes u octágonos) al no reflejar el comportamiento estocástico y caótico de un incendio real[28].
- **Modelos estocásticos empíricos (Autómata de Rothermel / Alexandridis):** Fusionan el modelo físico continuo de Richard Rothermel (estándar de la industria) con la adaptación matricial probabilística de Alexandridis. Estos modelos, En lugar de usar tiempos fijos, traducen las complejas ecuaciones físicas en probabilidades de ignición discretas (P_{ign}) para cada celda, introduciendo una aleatoriedad controlada que rompe la rigidez de la cuadrícula.

Tras analizar estas alternativas, se concluye que la aproximación estocástica basada en **Rothermel y Alexandridis** es la opción idónea para el simulador ya que resuelve los defectos de los modelos anteriores. Aporta el rigor físico del que carece la percolación y, gracias a su factor probabilístico, elimina los artefactos geométricos de los modelos deterministas logrando un avance orgánico. Además, su formulación matemática permite descomponer el cálculo en factores multiplicativos independientes, lo cual encaja a la perfección con la arquitectura de la tarjeta gráfica, permitiendo procesar la propagación de forma masiva.

Discretización espacial y regla de vecindad

El espacio lógico de simulación se abstrae mediante una cuadrícula tridimensional uniforme, donde la unidad mínima de información es el *voxel* (el equivalente volumétrico de un píxel). Esta discretización espacial permite asignar a cada coordenada de la matriz un único estado discreto, el cual define su naturaleza física y su comportamiento frente a las reglas del modelo.

En el contexto del simulador, el espacio se inicializa categorizando cada celda en dos grandes grupos: elementos inertes (como aire, subsuelo o tierra, que actúan como barreras) y elementos combustibles. Estos últimos se clasifican según su carga forestal (por ejemplo, matorral para vegetación baja o árbol para vegetación alta), poseyendo cada uno distintas resistencias térmicas y pudiendo cambiar dicho estado a fuego.

La evolución del incendio se decide evaluando los estados de las celdas vecinas y los factores ambientales. Para garantizar una propagación fluida, el algoritmo evalúa, para cada celda de combustible con estado intacto, una vecindad tridimensional de Moore la cual consiste en iterar un rango matricial de 3x3x3 alrededor de la celda de origen (descartando la posición central), y evaluando el estado de las 26 celdas adyacentes para calcular el riesgo total de ignición.

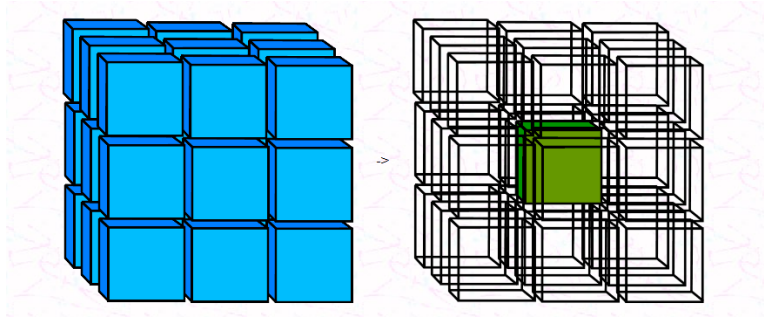


Figura 3.1: Modelo de la vecindad de Moore alrededor de la celda origen

Integración del modelo físico de propagación

El paso de un voxel intacto a una voxel en llamas sigue una adaptación estocástica del modelo físico de incendios de Rothermel (1972) aplicada a Autómatas Celulares, basándose en la formulación de Alexandridis. En este enfoque, la probabilidad de ignición (P_{ign}) transmitida por una celda vecina en combustión se define mediante la siguiente ecuación matemática[21][4]:

$$P_{ign} = P_0 \cdot (1 + p_w) \cdot (1 + p_s) \cdot p_h$$

Esta formulación divide la termodinámica de Rothermel en multiplicadores independientes que pueden procesarse en paralelo para millones de voxeles. Cuando una voxel que tenga la posibilidad de arder se encuentra junto a otros que ya están en estado de fuego, el sistema procesa el riesgo de contagio evaluando cada uno de estos factores:

- **Probabilidad base y propensión del combustible (P_0):** Representa el tipo y la carga de combustible. Se asignan diferentes pesos según la densidad de la vegetación presente en la matriz. Por ejemplo el matorral (vegetación baja) es considerado un combustible altamente volátil que arde con rapidez por lo que va a tener una mayor propensión, mientras que el árbol (vegetación alta) tiene una mayor resistencia a la ignición, es decir, una propensión más baja.
- **Factor de viento (p_w):** La influencia del viento se calcula penalizando o bonificando la probabilidad de propagación según si el viento sopla a favor o en contra de la celda y la velocidad del mismo. Se define como:

$$p_w = \exp(V \cdot (\cos(\theta) - 1))$$

Donde V es la velocidad del viento y θ es el ángulo formado entre la dirección del viento y la línea que une el voxel en llamas con su vecino. En la implementación, este factor afecta acelerando la propagación a favor del viento y bajando la probabilidad cuando se encuentre a contraviento.

- **Factor topográfico o de pendiente (p_s):** Representa el ascenso del aire caliente, el cual precalienta el combustible situado cuesta arriba. Su formulación es:

$$p_s = \exp(a \cdot \tan(\theta_s))$$

Donde θ_s es el ángulo de inclinación del terreno. El algoritmo incrementa exponencialmente el riesgo cuando el fuego proviene de un voxel inferior, y lo penaliza en el caso contrario, cuando intenta propagarse de una delta superior a una inferior.

- **Factor de humedad ambiental (p_h):** Funciona como freno general de la propagación ya que el agua contenida en la vegetación debe evaporarse antes de alcanzar el punto de ignición. Se modela como un decaimiento exponencial:

$$p_h = \exp(-c_h \cdot H)$$

Donde H es el índice de humedad relativa (de 0 a 1) y c_h es la constante de sensibilidad del combustible.

Cabe destacar el diseño estructural de la ecuación final. Los factores de viento y pendiente se añaden a la fórmula como multiplicadores de forma $(1 + p_w)$ y $(1 + p_s)$ debido a que actúan como ayudas adicionales, es decir, si las condiciones son favorables aumenta la probabilidad de propagación pero si en cambio van en contra los factores tienden a cero y el multiplicador interno queda en 1, permitiendo que el fuego siga propagándose solo con la probabilidad base (P_0).

Por el contrario, el factor de humedad (p_h) se aplica de forma directa porque actúa como un freno general. Si el entorno está empapado ($p_h \approx 0$), al multiplicar toda la ecuación por cero el fuego se detiene prácticamente por completo, reflejando la dinámica real de los incendios forestales.

Finalmente durante la iteración de la vecindad espacial, el algoritmo suma el riesgo individual calculado de cada una de las celdas vecinas activas, dando como resultado un valor global de riesgo de ignición para la celda evaluada. Teniendo este valor de probabilidad se calcula un número aleatorio y se determina si se cambia el estado a fuego o no.

3.2.4. Construcción del modelo volumétrico: Raymarching y doble pase

Una vez que la lógica de simulación está resuelta matemáticamente el siguiente reto arquitectónico es mostrar visualmente dicha matriz tridimensional. Intentar representar con mallas cada celda en llamas colapsaría el rendimiento del sistema por lo que se ha optado por utilizar un contenedor volumétrico el cual actúa como un lienzo tridimensional encargado de interpretar la matriz térmica calculada por el *Compute Shader*.

Para lograr esta representación se ha implementado la técnica de *Raymarching* (trazado de rayos volumétrico) que consiste en emitir un rayo virtual desde la cámara que avanza a través del interior del cubo paso a paso, muestreando en cada punto un campo de datos y así acumular color y densidad. Para determinar la trayectoria exacta que debe recorrer cada rayo, el sistema necesita conocer exactamente en qué coordenada espacial entra el rayo al cubo (cara frontal) y en qué coordenada sale (cara trasera)[14].

Antes de pasar a la explicación de la construcción de la escena es necesario comprender el concepto del *Frame Buffer Object* (FBO). Normalmente la GPU calcula la geometría y los shader y lo vuelca en un buffer que posteriormente se muestra por pantalla, pero al usar un FBO permite que dichos datos pasen a guardarse en la VRAM sin llegar a mostrarse por pantalla. Esto en Godot se consigue con el nodo `SubViewport` que actúa

como un entorno de renderizado aislado que almacena su resultado en una `ViewportTexture` que se puede usar posteriormente.

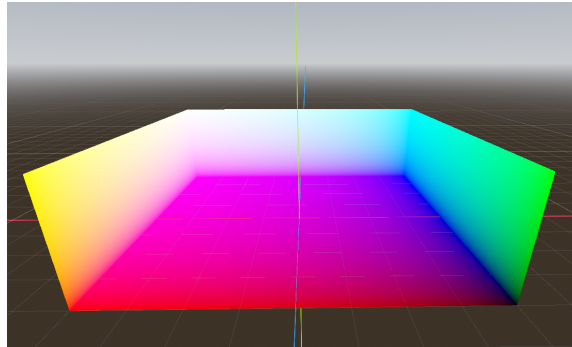


Figura 3.2: Vista de las caras traseras obtenidas por el FBO

Construcción de la escena y arquitectura de *Shaders*

Para obtener estas coordenadas espaciales simultáneamente, la escena se ha estructurado utilizando un paradigma de renderizado de doble pase, el cual requiere una configuración específica de cámaras, capas de renderizado y texturas:

- **Cámaras y sincronización:** El sistema despliega dos cámaras. Una cámara principal que se encarga de registrar el movimiento del usuario e interactuar con el entorno mientras que una segunda cámara se encuentra encapsulada dentro del entorno aislado de un nodo `SubViewport` que se utilizará posteriormente para la captura de una textura. Mediante un *script* en la cámara principal (y apoyado por nodos de transformación como el `RemoteTransform3D`), el sistema sincroniza la posición de ambas cámaras para que siempre muestren la misma geometría.
- **Contenedores volumétricos (Cubos):** Existen dos mallas en la escena, el `CuboFrontal` y el `CuboTrasero` (este último dentro del `SubViewport`). Para garantizar que los cálculos espaciales del rayo sean matemáticamente perfectos, ambos nodos comparten exactamente el mismo recurso de malla, asegurando que tengan la misma geometría y dimensiones en todo momento.
- **Aislamiento por capas (*Cull Masks*):** Para evitar interferencias visuales entre ambos pases de renderizado, se emplea una estrategia de exclusión mediante las máscaras de capa. El `CuboFrontal` reside en la capa visual 1, mientras que el `CuboTrasero` se ubica en la capa 2. En lugar de restringir las cámaras a una única capa, se configuran

para procesar todo el entorno excluyendo únicamente la capa del objeto opuesto, haciendo que la cámara secundaria tenga deshabilitada la capa 1 (ocultando así el cubo frontal), y la principal tenga deshabilitada la capa 2 (ocultando la geometría trasera). Esta configuración garantiza que cada cámara calcule exclusivamente el volumen que le corresponde sin dejar de percibir el resto de la escena.

- **Configuración de *Shaders* y *Render Modes*:** El núcleo de esta técnica recae en cómo la GPU procesa la superficie de estos cubos. El **CuboTrasero** posee un *shader* configurado con el parámetro de renderizado `cull_front` que obliga al motor a descartar las caras delanteras de la geometría y renderizar únicamente las caras internas (traseras). Este resultado, que contiene las coordenadas espaciales de las caras traseras, es capturado por el **SubViewport** en una textura dinámica que posteriormente es aceptada como parámetro por el shader que posee el **CuboFrontal**. Este último shader está configurado a la inversa del primero, es decir, posee el parámetro (`cull_back`) que renderiza solo el exterior del volumen y pueda así ser usado para ejecutar el *Raymarching*[29].

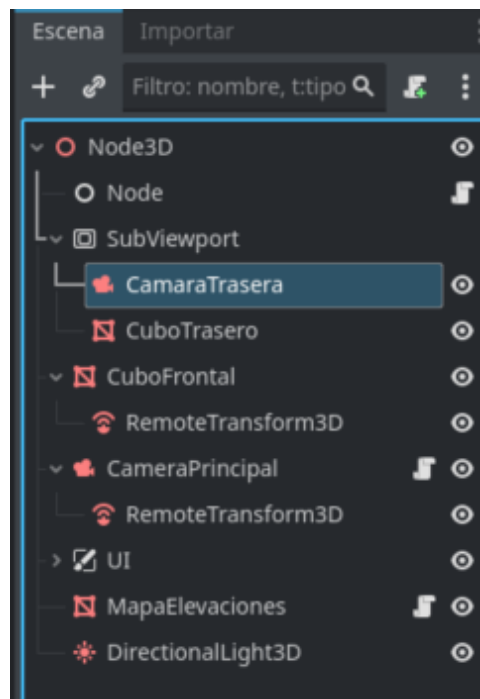


Figura 3.3: Esquema de nodos de la escena del proyecto en Godot 4

Flujo técnico del renderizado de doble pase

El flujo técnico de este contenedor volumétrico opera de la siguiente manera:

1. **Primer pase (FBO y caras traseras):** El entorno de renderizado del `SubViewport` utiliza una cámara interna, sincronizada con la cámara principal del usuario, y un cubo que comparte la misma malla que el cubo en la escena principal. Para renderizar únicamente las caras traseras se usa un *shader* que se encarga de ocultar las delanteras y ajustar la posición a un color RGB válido (de 0.0 a 1.0). Esto es necesario ya que como se mencionó anteriormente la información del `SubViewport` se guarda en una `ViewportTexture`
2. **Segundo pase (Caras frontales y cálculo del rayo):** En la escena principal, al igual que en el `SubViewport`, se encuentran la cámara principal y el cubo frontal. Este último posee un *shader* que al contrario del anterior oculta las caras traseras y lee las frontales visibles. Además acepta como parámetro la textura generada por el `SubViewport` para que por cada fragmento al restar la posición de la cara frontal a la trasera se pueda obtener el vector del rayo y saber así la dirección exacta en la que atraviesa el volumen y la distancia total de recorrido.
3. **Avance e interpolación de la matriz (UVW):** Una vez definido el rayo, el algoritmo divide la distancia total en un bucle con un número de pasos definidos por un parámetro y con un tamaño de paso que se calcula teniendo en cuenta este número de pasos y el tamaño total del rayo. Además en cada paso, la posición local actual del rayo, que oscila entre -0.5 y 0.5, se normaliza desplazándola al rango de coordenadas de textura de 0.0 a 1.0.
4. **Muestreo y acumulación de densidad:** Utilizando las coordenadas normalizadas calculadas en cada iteración del bucle, el *shader* consulta el estado de ese voxel en la textura que le ha llegado como parámetro desde el *Compute Shader*, donde un valor de 1.0 indica la existencia de fuego y 0.0 indica espacio vacío. Si el rayo atraviesa un voxel con fuego, se incrementa un contador el cual se usa posteriormente para calcular la densidad de ese fragmento.
5. **Generación del espectro térmico:** Finalmente, con el valor de la densidad acumulada a lo largo del rayo se determina el color del píxel resultante en pantalla, para ello el sistema aplica una interpolación lineal entre colores anaranjados para las zonas de densidad baja y colores amarillos intensos para las zonas de alta densidad.

3.2.5. Comunicación y Sincronización CPU-GPU

Como se ha mencionado anteriormente, el costoso cálculo de la probabilidad de cada voxel para la propagación pasará a hacerse en un *Compute Shader* para así aprovechar la alta capacidad de procesamiento paralelo de la GPU. Trabajar entre los dos espacios de memoria (la RAM y la VRAM) requiere una sincronización y transferencia eficiente de datos, que se logra de una manera más sencilla haciendo uso de la clase `RenderingDevice`.

El ciclo de comunicación y ejecución de la simulación se divide en tres fases: la inyección de datos paramétricos, la sincronización de estados mediante *Ping-Pong Buffers* y el despacho masivo de hilos.

Inyección de datos estáticos y parámetros dinámicos

Antes de iniciar el cálculo de la propagación, la CPU debe preparar y transferir el contexto del entorno a la GPU. El flujo de datos se divide en dos categorías según su frecuencia de actualización: estáticos y dinámicos.

Los **datos estáticos** son la topografía y el tipo de vegetación y se evalúan una única vez durante la inicialización del sistema. El *script* generador del terreno recorre la matriz de elevaciones y, en función de la clasificación por colores, construye un `PackedByteArray` que se transfiere a la VRAM, donde el `RenderingDevice` lo ensambla como una textura 3D vinculada al *shader*.

```

1      # Generacion y envio del mapa estatico (mapa_elevaciones.
      gd)
2      func enviar_mapa_combustibles_a_gpu():
3          var array_combustibles = PackedByteArray()
4          array_combustibles.resize(total_celdas)
5
6          # ... (bucle tridimensional X, Y, Z) ...
7          if estado == ESTADO_MATORRAL:
8              array_combustibles[indice_gpu] = 100
9          elif estado == ESTADO_ARBOL:
10             array_combustibles[indice_gpu] = 255
11
12             # Se inyecta en la GPU a traves del controlador
13             controlador.cargar_mapa_combustibles(
                array_combustibles)

```

Por otro lado, los **datos dinámicos** (velocidad del viento, dirección y porcentaje de humedad) se actualizan en tiempo real durante la ejecución. Para lograr esto, se hace uso de *Push Constants*, una técnica que permite enviar pequeños bloques estructurados de memoria directamente a los registros ultrarrápidos de la GPU.

```

1      # Inyeccion de datos dinamicos por frame (controlador.gd)
2      func ejecutar_compute_shader():

```

```

3      # Empaquetamos tiempo (semilla) y meteorología en 32
      bytes exactos
4      var push_constant = PackedFloat32Array([
5          semilla_cpu, 0.0, 0.0, 0.0,
6          velViento, dir_rad, humedad, 0.0
7      ])
8      var bytes = push_constant.to_byte_array()
9
10     # Inyectamos en los registros de la GPU antes de
      ejecutar
11     rd.compute_list_set_push_constant(compute_list, bytes,
      bytes.size())

```

Sincronización de estado: Técnica de *Ping-Pong Buffers*

El modelo de propagación del autómatas celular presenta un problema, si un hilo actualiza el estado de una celda de intacta a ardiendo y sobrescribe la matriz instantáneamente, los hilos que aún no hayan finalizado su ciclo leerán el dato ya modificado lo que generaría una condición de carrera que corrompería la validez del modelo.

Para solucionar este conflicto, se implementa la técnica de *Ping-Pong Buffers*, que en lugar de utilizar una única matriz, el sistema reserva dos texturas tridimensionales idénticas en la VRAM rellenándolas inicialmente con ceros que consiste en configurar un *buffer* (el estado anterior) en modo de solo lectura, mientras que el segundo (el estado nuevo) como solo escritura.

- **Buffer A :** Configurado en modo de solo lectura (**readonly**). Contiene la "fotografía" estática del estado actual del bosque que los hilos utilizan para evaluar a sus vecinos.
- **Buffer B:** Configurado en modo de solo escritura (**writeonly**). Es el lienzo en blanco donde cada hilo plasma el resultado de sus cálculos para el siguiente instante de tiempo.

Al finalizar cada orden de despacho computacional, la CPU invierte los punteros lógicos de ambas texturas para el siguiente *frame*.

```

1      # Intercambio de buffers por iteracion (controlador
      .gd)
2      if buffer_a_es_lectura:
3          rd.compute_list_bind_uniform_set(compute_list,
          set_a_lee_b_escribe, 0)
4      else:
5          rd.compute_list_bind_uniform_set(compute_list,
          set_b_lee_a_escribe, 0)
6
7      # ... (Despacho a la GPU) ...
8
9      # Preparamos el siguiente turno invirtiendo los
      roles
10     buffer_a_es_lectura = !buffer_a_es_lectura

```

Configuración de *Workgroups* y Despacho (*Dispatch*)

El último paso de la arquitectura es la ejecución paralela del código (*Compute Shader*). El volumen no se procesa como una lista unidimensional, sino que se subdivide espacialmente para adaptarse a la arquitectura de la tarjeta gráfica y así aprovechar el paralelismo que nos ofrece.

El *Compute Shader* está configurado para agrupar las invocaciones en clústeres cúbicos de $8 \times 8 \times 8$ hilos por grupo. Durante la ejecución, la CPU calcula la cantidad de grupos necesarios dividiendo la resolución total de la matriz entre tamaño del *Workgroup* en cada uno de sus ejes. Finalmente, se envía el comando de despacho (*Dispatch*) a la GPU que instancia simultáneamente los bloques resolviendo la propagación.

```
1      # Division espacial y despacho concurrente (
2      controlador.gd)
3      var grupos_x = ceil(TAMANO_GRID.x / float(
4      TAMANO_WORKGROUP.x))
5      var grupos_y = ceil(TAMANO_GRID.y / float(
6      TAMANO_WORKGROUP.y))
7      var grupos_z = ceil(TAMANO_GRID.z / float(
8      TAMANO_WORKGROUP.z))
9
10     # Orden de ejecucion a la GPU
11     rd.compute_list_dispatch(compute_list, grupos_x,
12     grupos_y, grupos_z)
13     rd.compute_list_end()
```

3.3. Geometría y Datos Topográficos

Un punto clave para un simulador de incendios es tener una buena representación del entorno para así poder conseguir resultados rigurosos. En este proyecto, la geometría y la distribución del combustible se generan de forma procedimental a partir de Modelos Digitales de Elevaciones (DEM) y mapas satelitales de vegetación[3].

3.3.1. Procesamiento SIG, normalización y adecuación de formatos

Los Modelos Digitales de Elevaciones obtenidos a través de organismos oficiales se encuentran normalmente en formato GeoTIFF (*.tif*). Aunque este formato es idóneo para Sistemas de Información Geográfica (SIG), su importación directa a motores de renderizado 3D suele presentar numerosos problemas tanto en la lectura como en la precisión debido a limitaciones en la profundidad de bits por canal.

Para solucionar estos problemas y garantizar un relieve topográfico y de buena fidelidad, se necesitó usar el software QGIS [2], mediante el cual los mapas originales fueron reproyectados espacialmente y delimitados al área de interés del simulador.

Un paso crítico en esta fase fue la normalización matemática de los valores. Las cotas geográficas reales se normalizaron en un rango de 0.0 a 1.0 para que el *shader* encargado de generar la elevación del terreno procese los valores como una variable de color escalar.

Finalmente, el resultado es exportado al formato OpenEXR (.exr). Este formato, un estándar en la industria de los gráficos por ordenador, soporta valores de coma flotante lineales de 32 bits, asegurando que el mapa normalizado no sufra pérdida de precisión decimal antes de inyectarse en el motor gráfico.

3.3.2. Renderizado del terreno: Desplazamiento y normales

Una vez procesados, los mapas se integran en el simulador. La representación visual no emplea una malla estática, sino que parte de un plano base que es deformado mediante un *shader* espacial (*spatial*)[24].

Este *shader* se encarga de alterar la coordenada vertical de cada vértice muestreando el mapa de elevaciones introducido. Posteriormente, el valor normalizado leído se multiplica por el diferencial real de altura del terreno, obteniendo así la elevación exacta del relieve original.[30]

```
1      // Lectura y desplazamiento geometrico de la orografia
      (Shader)
2      void vertex() {
3          // Lectura de la altitud normalizada (0.0 a 1.0) en
           el mapa OpenEXR
4          float altura_leida = textureLod(mapa_elevacion, UV,
           0.0).r;
5
6          // Proteccion matematica contra pixeles corruptos (
           NaN/Inf)
7          float altura_segura = (isinf(altura_leida) || isnan
           (altura_leida))
8              ? altura_base : altura_leida;
9
10         // Decodificacion: Desplazamiento 3D escalar del
           vertice
11         VERTEX.y += (altura_segura - altura_base) *
           escala_altura;
12
13     }
```

Un problema generado por alterar los vértices de forma dinámica es que los vectores normales originales del plano dejan de ser válidos, lo que provoca que no se calcule correctamente las sombras y la iluminación. Para solucionarlo, el *shader* recalcula la inclinación real del terreno leyendo la altitud de los cuatro píxeles vecinos y construyendo dos vectores direccionales (el vector tangente y el vector binormal) cuya diferencia angular conforma la pendiente[1].

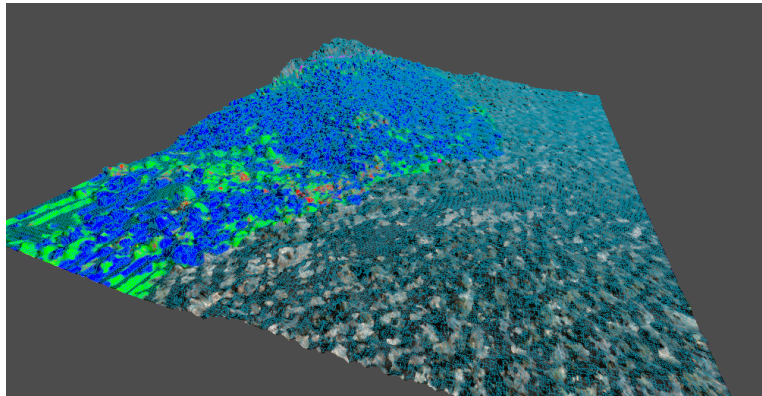


Figura 3.4: Mapa generado a partir de un Modelo Digital de Elevaciones

Finalmente, el *shader* le asigna el color leyendo el mapa satelital de vegetación y ajustando los parámetros físicos del material (ROUGHNESS y SPECULAR) para simular la reflexión lumínica parecida a la de la realidad.

3.3.3. Extracción a memoria principal y discretización volumétrica

Para construir la matriz lógica del autómata celular, la CPU extrae, descomprime y redimensiona las texturas visuales hacia la memoria RAM. Este volumen espacial se estructura en un (`PackedInt32Array`) para obtener un mejor rendimiento y se procesa siguiendo un proceso:

1. **Cálculo espacial:** Lee el píxel del mapa de elevaciones y lo multiplica por la resolución vertical del contenedor volumétrico para calcular índice exacto de la superficie.
2. **Análisis del color vegetal:** Analiza los canales RGB del píxel. Si el espectro azul es dominante, se almacena como vegetación alta (`ESTADO_ARBOL`), si predomina el rojo, vegetación baja (`ESTADO_MATORRAL`) y el resto de valores se asume como terreno que no puede incendiarse (`ESTADO_SUBSUELO`)

Finalmente, el algoritmo rellena verticalmente los voxels inferiores al que contiene la vegetación como voxels con estado (`ESTADO_SUBSUELO`)

3.4. Interacción y Control Dinámico

Para que el simulador sea accesible y permita al usuario visualizar la escena desde distintos ángulos e interactuar con los factores de simulación se han desarrollado una serie de mecanismos y elementos en Godot.

3.4.1. Generación de colisiones topográficas

El renderizado de la simulación volumétrica y el *shader* que genera el desplazamiento topográfico de las elevaciones no poseen una malla para el subsistema de colisiones de Godot, impidiendo que se pueda interactuar con el terreno mediante el ratón.

Para solucionar esto se ha generado una `HeightMapShape3D` que es un componente que construye dinámicamente un campo de alturas sólido superpuesto a la representación visual. Los valores de altitud originales se ponderan por el factor de diferencia escalar y se ajustan a las cotas límite del mundo. Una vez calculada esta malla de colisión se pueden llevar a cabo los (*RayCasts*) emitidas desde la cámara del usuario para desencadenar la ignición inicial del incendio de forma precisa e interactiva.

3.4.2. Navegación espacial y cámara libre

La simulación cuenta con un sistema de cámara de vuelo libre, que permite al usuario desplazarse sin restricciones espaciales por la escena, utilizando un control clásico mediante teclado y ratón.

La capacidad de observar la propagación de las llamas desde cualquier punto resulta de gran utilidad ya que permite ver en detalle cómo el fuego interactúa con pendientes y tipos de vegetación específicos y obtener así una perspectiva del alcance general de la emergencia.

3.4.3. Sistema de Ignición: Interacción espacial y traducción

Además de la visualización de con el movimiento de la cámara se ha implementado un sistema (*RayCast*) que interactúa con la malla de colisión generada para el terreno y que nos permite iniciar el incendio en cualquier punto de la topografía.

Al registrarse el clic de ignición, el motor proyecta un rayo desde el origen de la cámara en movimiento hacia el mundo físico. Si este rayo intersecta con la malla de colisión topográfica generada anteriormente, el algoritmo recupera el punto de impacto global.

1

```
# Lanzamiento del rayo de interaccion (mapa_elevaciones
.gd)
```

```

2      var centro_pantalla = get_viewport().get_visible_rect()
        .size / 2.0
3      var origen = camara.project_ray_origin(centro_pantalla)
4      var direccion = camara.project_ray_normal(
        centro_pantalla)
5
6      # Proyeccion e interseccion fisica
7      var impacto = espacio_fisico.intersect_ray(
        parametros_rayo)

```

Debido a que la matriz topográfica opera en un espacio local, la coordenada global del impacto debe ser traducida al sistema de referencia del volumen contenedor. Primero normaliza los vectores para evitar desfases de escala, y seguidamente el algoritmo calcula los índices matemáticos discretos (X, Z) multiplicando la posición local por la resolución base de la matriz.

```

1      # Traduccion a matriz y encendido espacial (
        mapa_elevaciones.gd)
2      var pos_local = cubo_frontal.to_local(punto_global)
3      var pos_normalizada = pos_local + Vector3(0.5, 0.5,
        0.5)
4
5      # Calculo del indice matricial (X, Z)
6      var x_matriz = int(pos_normalizada.x * resolucion.x)
7      var z_matriz = int(pos_normalizada.z * resolucion.z)
8
9      # El controlador actualiza la VRAM con el nuevo foco
10     controlador.agregar_fuego_en(x_matriz, y_matriz,
        z_matriz)

```

3.4.4. Interfaz de Usuario y Control Meteorológico

En un entorno real durante un incendio los factores ambientales no son estáticos sino que cambian a lo largo del tiempo que dura el incendio. En el simulador se ha logrado esto mediante una Interfaz de Usuario (UI) superpuesta al visor volumétrico.

Dicha interfaz tiene varios *sliders* representando la velocidad del viento, dirección del viento (en grados) y el índice de humedad. Estos *sliders* están vinculados mediante señales al *script* principal del controlador por lo que cualquier modificación realizada por el usuario es capturada instantáneamente.

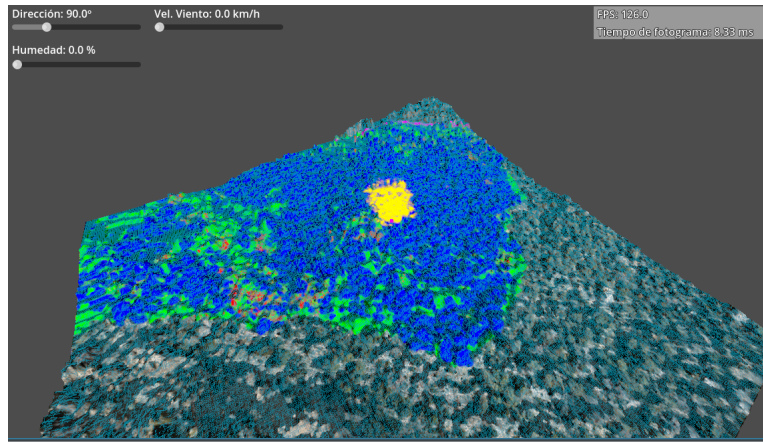


Figura 3.5: Interfaz de Usuario del simulador

Dado que estos parámetros se envían a la tarjeta gráfica empaquetados como *Push Constants* en cada ciclo de iteración, el cambio en las condiciones del entorno tiene un efecto inmediato en el *Compute Shader*, lo que permite observar cómo las llamas se adaptan a las nuevas variables sin necesidad de pausar, recalcular o reiniciar la simulación.

3.5. Desafíos Técnicos y Soluciones

El desarrollo de este simulador ha supuesto diversos retos arquitectónicos y matemáticos. A continuación, se detallan los desafíos más significativos encontrados durante la implementación y las soluciones aplicadas para resolverlos.

3.5.1. Optimización del *Raymarching* mediante Salida Temprana

El problema: La técnica de *Raymarching* tiene un alto coste computacional. Para una resolución de pantalla estándar (1080p), emitir un rayo por cada píxel y hacerlo dar un gran número de pasos implicaba ejecutar decenas de millones de lecturas a la textura 3D por cada *frame*. Inicialmente, se evaluaba el rayo completo incluso cuando atravesaba zonas densas de fuego o zonas completamente vacías, provocando bajos fotogramas por segundo (FPS).

La solución: Se implementó una estrategia algorítmica de salida temprana en la que en *shader* de la segunda pasada, se estableció un umbral para la densidad acumulada. Al alcanzar este umbral el bucle se interrumpe

eliminando los cálculos innecesarios en las regiones internas y posteriores del fuego, y estabilizando el rendimiento en tiempo real.

```
1      // Optimizacion de salida temprana (TwoPassRendering.  
      gdshader)  
2      densidad_acumulada += (densidad_punto * multiplicador)  
      * tamano_paso;  
3  
4      if (densidad_acumulada >= 1.0) {  
5          break; // El rayo no necesita seguir avanzando  
6      }
```

3.5.2. Sincronización de Espacios de Coordenadas

El problema: El sistema de ignición interactiva presentaba una desincronización espacial. Al realizar un clic sobre una montaña, el fuego aparecía en ubicaciones incorrectas o fuera de los límites de la matriz. Esto ocurría porque no se estaban haciendo correctamente los cambios entre los tres espacios matemáticos, las coordenadas globales del motor físico, las coordenadas locales del cubo contenedor y el índice discreto de matriz del autómata celular.

La solución: Se desarrolló un flujo de traducción de coordenadas secuencial. Primero, el punto de impacto global devuelto por el *RayCast* se transforma al espacio local del contenedor. Debido a que las coordenadas de la malla se encuentran entre -0.5 y 0.5, se le suma un vector (0.5, 0.5, 0.5) para obtener coordenadas normalizadas estrictamente positivas entre 0.0 y 1.0. Finalmente, este valor decimal se multiplica por la resolución máxima de la matriz del autómata

3.5.3. Anomalías Geométricas en la Generación del Relieve

El problema: Durante las primeras pruebas de importación de los mapas de elevaciones (DEM), el desplazamiento de los vértices en el *shader* generaba columnas gigantes y picos que no coincidían con la topografía real. Investigandolo se descubrió que este fallo se originaba por dos factores. En primer lugar, los datos del DEM representaban altitudes reales en metros que producían un desbordamiento que disparaba los vértices fuera de los límites de renderizado. En segundo lugar, al recortar las regiones de interés en QGIS, algunos píxeles se exportaban con valores nulos (NaN), o infinitos (Inf).

La solución: Para solucionar el primer problema se optó por la normalización de la textura en QGIS. Mediante el uso de álgebra de mapas, todas las cotas se remapearon a un rango de 0.0 a 1.0, permitiendo que el despla-

zamiento de vértices usando un escalado para la altura de como resultado una visualización correcta.

Para el segundo problema se aplicó directamente un filtro en el *shader* que se encarga de evaluar las alturas leídas de la textura normalizada mediante las funciones nativas `isinf()` e `isnan()`. Si el píxel es nulo o infinito, fuerza su altitud a la altura base.

```

1 // Proteccion matematica contra pixeles corruptos
2 float altura_leida = textureLod(mapa_elevacion, UV, 0.0).r;
3
4 // Si la imagen EXR tiene errores (Infinito o NaN),
5 // forzamos la altura base
6 float altura_segura = (isinf(altura_leida) || isnan(
7     altura_leida))
8     ? altura_base : altura_leida;
9
10 // Desplazamiento 3D proporcional tras la normalizacion en
11 // QGIS
12 VERTEX.y += (altura_segura - altura_base) * escala_altura;

```

3.5.4. Desincronización de Cámaras en el Contenedor Volumétrico

El problema: Cuando el usuario utilizaba la cámara libre para moverse por el escenario ocurrían varios errores en la visualización del fuego, como que el volumen del fuego aparecía cortado o que flotaba fuera de su posición. Tras investigarlo se descubrió que el origen del error se daba en la técnica de doble pase, más concretamente en la sincronización de las posiciones. Para que el cálculo del vector del rayo sea correcto la cámara secundaria del `SubViewport` debe observar el cubo desde el mismo ángulo exacto que la cámara principal.

La solución: Para solucionar estos errores se desarrolló un *script* que garantiza que al inicio de la simulación la cámara secundaria copie los parámetros de proyección de la cámara principal, es decir, el *FOV*, el *Near* y el *Far*.

Por otro lado, se usó un nodo que proporciona Godot, el (`RemoteTransform3D`), sobre la cámara principal vinculándolo con la cámara del `SubViewport`. Este nodo se encarga de que la cámara secundaria replique continuamente la posición, rotación y escala de la principal.

Al sincronizar perfectamente ambas cámaras, las coordenadas de textura proyectadas en el `SubViewport` coinciden sin errores, haciendo que se visualice correctamente desde cualquier ángulo y distancia.

3.6. Integración y uso de Inteligencia Artificial

Durante el desarrollo de este proyecto, se ha hecho uso de modelos de Inteligencia Artificial (IA) generativa, concretamente **Gemini** apoyado por **Claude** para las labores específicas de programación, como asistentes para diversas tareas. El uso de esta tecnología se ha planteado desde un punto de vista instrumental, como una herramienta más, con el objetivo de optimizar los tiempos de aprendizaje, resolución de problemas y redacción, manteniendo en todo momento la autoría, el diseño arquitectónico y el criterio técnico sobre el resultado final.

Al enfrentarse a una curva de aprendizaje pronunciada con herramientas complejas nunca antes vistas, como el motor de desarrollo Godot 4 o el software de información geográfica QGIS, Gemini ha actuado como un tutor interactivo acelerando la comprensión de los flujos de trabajo de los entornos y las herramientas que estos ofrecían. Por otro lado, uno de los mayores retos del proyecto fue orquestar una comunicación eficiente entre la memoria del sistema (RAM) y la memoria de vídeo (VRAM). En este ámbito analítico, Claude sirvió como soporte de consulta para entender a bajo nivel el funcionamiento de la clase `RenderingDevice`, asimilar los conceptos de despacho concurrente en los *Compute Shaders* y agilizar la escritura de código y depuración de dicha sincronización.

A su vez, la representación visual de la matriz térmica mediante el contenedor volumétrico supuso un gran desafío técnico durante sus etapas iniciales debido a los errores que iban apareciendo. En este contexto de desarrollo, la IA demostró ser un recurso muy eficaz para el escrutinio de código y el diagnóstico rápido de fallos, permitiendo aislar y corregir de forma ágil los distintos *bugs* y anomalías visuales que corrompían la imagen, acelerando así la corrección de los *shaders* hasta alcanzar la fidelidad visual buscada en el simulador.

Finalmente, para la elaboración de esta memoria, Gemini ha desempeñado un papel fundamental como asistente para la estructuración y revisión de estilo de la misma. Se empleó principalmente para perfilar la estructura lógica de los capítulos, afianzar el rigor formal exigido en el ámbito universitario y facilitar el formateo del texto y las fórmulas matemáticas mediante $\text{\LaTeX}$. En definitiva, la inteligencia artificial ha actuado como un recurso complementario que ha agilizado significativamente todas las fases del trabajo. No obstante, la validación de los modelos físicos, las decisiones de ingeniería de software y la implementación final del sistema han dependido exclusiva y estrictamente del criterio del autor.

Capítulo 4

Resultados y Validación

4.1. Entorno y Metodología de Evaluación

El análisis y evaluación de un simulador que combina renderizado volumétrico y cálculo intensivo mediante *Compute Shaders* requiere definir el entorno de ejecución. Puesto que uno de los objetivos es garantizar la viabilidad del simulador en sistemas portátiles, las pruebas se han realizado sobre un equipo informático móvil (un ordenador portátil) con especificaciones estándar de mercado.

Entorno de Hardware y Software

Todas las simulaciones, extracciones de métricas y capturas de rendimiento se han ejecutado bajo el siguiente entorno:

- **Procesador (CPU):** AMD Ryzen 7 4800H a 2.9 GHz.
- **Memoria RAM:** 16 GB DDR4.
- **Procesador Gráfico (GPU):** NVIDIA GeForce GTX 1650 con 4 GB de memoria VRAM dedicada.
- **Sistema Operativo:** Windows 11 Home de 64 bits.
- **Motor Gráfico y API:** Godot Engine 4.5 bajo la interfaz de renderizado de alto nivel Vulkan (Forward+).

Métricas y Metodología de Evaluación

La validación del simulador no consiste solo en comprobar su correcto funcionamiento visual, sino que se han llevado a cabo varias pruebas de evaluación distintas para medir tanto su validez científica como su eficiencia computacional.

1. **Validación del modelo físico:** En estas pruebas se plantean escenarios de control aislados para observar cómo reacciona el fuego ante variaciones de la topografía (pendiente) y de las variables meteorológicas (velocidad, dirección del viento y humedad), comprobando si la propagación respeta las ecuaciones del modelo de Rothermel.
2. **Rendimiento y coste computacional:** Para estas pruebas se monitoriza la carga de trabajo del procesador gráfico (GPU), midiendo el coste en milisegundos (*ms*) de la ejecución del *Compute Shader* frente al renderizado del *Raymarching*. Además, se registrará la tasa de FPS para asegurar que mantiene una buena fluidez durante toda la simulación a pesar de la alta carga volumétrica.

Tras el análisis de los resultados de estas evaluaciones se podrá concluir si la solución desarrollada cumple con los requisitos para un sistema portátil de apoyo a la toma de decisiones en la lucha contra incendios forestales.

4.2. Validación del Modelo Físico de Rothermel

El objetivo de esta fase de pruebas es comprobar que la lógica del autómata celular responde de manera coherente a los principios térmicos establecidos por el modelo de Rothermel. Para que sea más sencillo de ver que funcione correctamente cada parámetro se han diseñado tres casos de estudio con variables ambientales y topografía diferentes.

Caso de Estudio A: Escenario Base

En este primer escenario, se establece una topografía completamente plana con una carga de combustible homogénea (vegetación alta) y ausencia total de viento (velocidad 0 km/h).

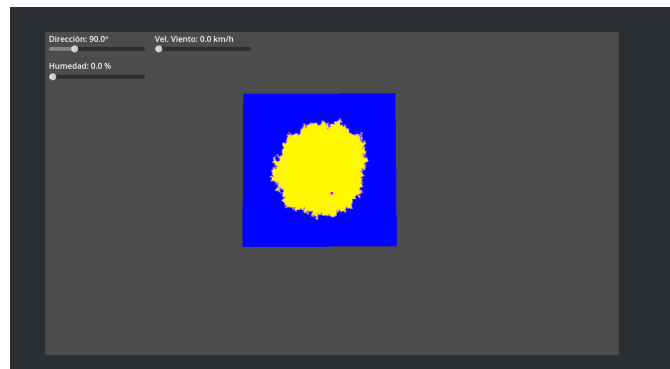


Figura 4.1: Propagación circular en terreno llano y sin viento.

Resultado observado: Cuando se desencadena el incendio en un punto con toda la vegetación homogénea y con ausencia de viento el modelo estocástico hace que el fuego se propague con una probabilidad muy parecida en todas las direcciones. El incendio dibuja un patrón de crecimiento circular a medida que avanza el tiempo de simulación. Esto valida que la matriz tridimensional del autómata no sufre de sesgos direccionales por la vecindad de Moore.

Caso de Estudio B: Influencia eólica y deformación elíptica

En el segundo escenario se mantiene un terreno plano con una combustible homogéneo de vegetación alta pero se introduce una variable meteorológica: rachas de viento constante a 40 km/h incidiendo desde un ángulo fijo.

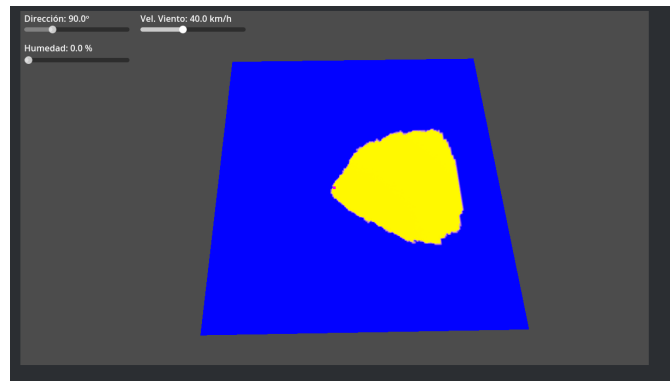


Figura 4.2: Deformación elíptica del incendio bajo la influencia de viento fuerte unidireccional.

Análisis de resultados: Como se observa en la *Figura X*, la simulación avanza de una manera realista siguiendo el factor del empuje del viento. El incendio deja de expandirse por igual en todas las direcciones y pasa a tener una deformación a favor de la dirección del aire que da como resultado una forma de elipse.

Al observar el punto de ignición original (el extremo izquierdo de la superficie amarilla) se comprueban dos fenómenos que produce el modelo de Rothermel:

- **Aceleración a favor del viento (Frente de avance):** En la dirección en la que sopla el viento la probabilidad de ignición aumenta propagándose en esa dirección con una mayor velocidad.
- **Estancamiento a contraviento (Fuego de retroceso):** El avance

del fuego en contra de la dirección del viento es prácticamente nulo. Esto valida el correcto funcionamiento y la ecuación de Rothermel con respecto a la realidad.

Caso de Estudio C: Influencia topográfica y convección

El tercer escenario evalúa el impacto del relieve, ignorando el viento. Se utiliza una geografía generada a partir de una textura de ruido con pronunciadas elevaciones y se realiza una prueba de doble ignición: el primer foco se inicia en la base de un barranco, y el segundo foco se establece en la cresta de una ladera.

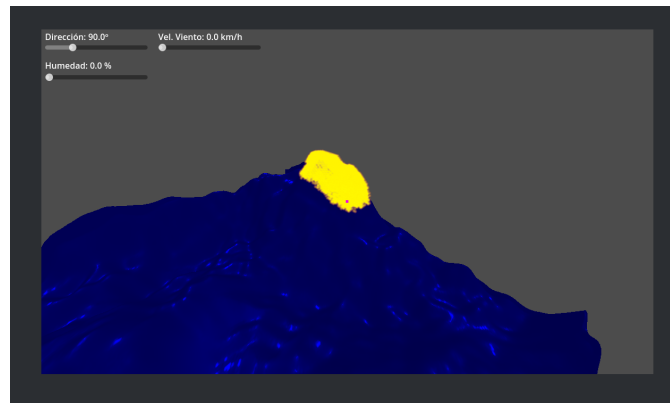


Figura 4.3: Comportamiento convectivo: aceleración del fuego ladera arriba y estancamiento en el descenso.

Resultado observado: El simulador replica el comportamiento físico de la transferencia de calor por convección. Cuando se origina un foco en mitad de una ladera se propaga rápidamente a las zonas superiores, siguiendo el factor de inclinación del modelo de Rothermel, mientras que presenta una enorme dificultad para la propagación a zonas inferiores.

Conclusión de la Validación Física

El análisis de los tres casos de estudio confirma que la implementación en el *Compute Shader* de los factores probabilísticos de viento (p_w) y pendiente (p_s) son correctos. Como resultado se puede concluir que el simulador es capaz de ofrecer una representación orgánica y fiel a las físicas del fuego sin caer en el crecimiento artificial en forma de ciertos patrones propios de autómatas celulares mal calibrados.

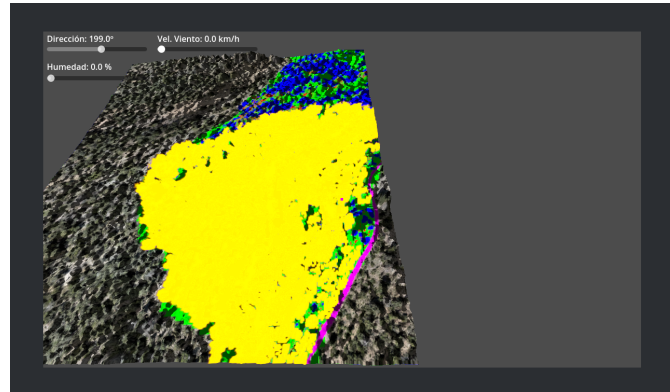


Figura 4.4: Comportamiento de la simulación con el mapa de elevaciones DEM.

4.3. Impacto del rendimiento a las distintas resoluciones

Para determinar el impacto de rendimiento se han realizado una serie de mediciones. En el primer experimento se ha evaluado el límite de estrés de la tarjeta gráfica para encontrar el punto óptimo entre una buena resolución de voxels y un buen rendimiento. Posteriormente, tras haber encontrado la resolución ideal, se ha evaluado y analizado el rendimiento en las distintas etapas de la propagación del incendio.

Estas pruebas se llevaron a cabo deshabilitando la sincronización vertical (V-Sync) y retirando los limitadores de velocidad de ejecución que se encuentran en el código, pudiendo medir así la capacidad bruta de procesamiento. Además para todos ellos se utilizó un mapa de elevaciones con tamaño 904x856 en el que toda la vegetación se cambió por vegetación alta para que no hubiese alteraciones.

Experimento 1: Escalabilidad espacial y límite de estrés

Como se ha mencionado anteriormente este experimento busca someter a la arquitectura a una prueba de estrés progresiva en la que se va aumentando el número de voxels de la matriz tridimensional comenzando con una resolución baja y llegando al límite de la tarjeta gráfica del dispositivo usado antes de saturarse por completo.

Resolución (X, Y, Z)	Total Vóxeles	Rendimiento (FPS)	Tiempo de Frame (ms)
$64 \times 20 \times 64$	81.920	308	2,78
$128 \times 40 \times 128$	655.360	293	3,30
$192 \times 60 \times 192$	2.21.840	267	3,40
$256 \times 80 \times 256$	5.242.880	230	4,05
$384 \times 120 \times 384$	17.694.720	176	6,14
$512 \times 160 \times 512$	41.943.040	113	9,09
$768 \times 200 \times 768$	117.964.800	53	18,30
$1024 \times 256 \times 1024$	268.435.456	16	65,28

Tabla 4.1: Evolución del rendimiento frente al incremento de la matriz volumétrica.

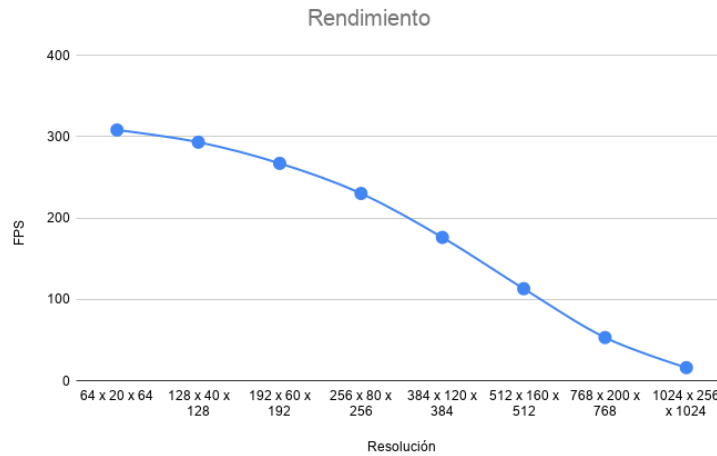


Figura 4.5: Curva de caída de rendimiento frente incremento de la matriz volumétrica.

Los datos anteriores (*Tabla 4.1* y *Figura 4.5*) muestran el buen rendimiento del simulador donde la GPU es capaz de realizar los cálculos y mantener una buena fluidez por encima de los 30 FPS para resoluciones con más de 100 millones de voxels. Solo cae en el caso de la más alta, 1024x256x1024, en la cual baja a 16 FPS en el momento de mayor estrés siendo esto ya demasiado bajo para el correcto funcionamiento de un simulador en tiempo real.

Sin embargo, durante las pruebas se comprobó que durante la fase de inicialización de la simulación había un cuello de botella que se comienza a notar con resoluciones a partir de 384x120x384. La CPU necesita un tiempo para leer los mapas topográficos, clasificar los tipos de combustible y alojar la matriz en la memoria RAM antes de transferirla a los *buffers* de la VRAM

ya que lo hace con un procesamiento secuencial y dicho tiempo puede variar entre unos pocos segundos, como en el caso de $384 \times 120 \times 384$ que necesita alrededor de 12 segundos, a unos pocos minutos en las resoluciones más altas.

Este fenómeno nos ayuda a entender la decisión arquitectónica de realizar los cálculos principales de la propagación en la GPU, ya que si el procesador se satura solo con recorrer e inicializar la matriz una vez, resultaría inviable que se encargase del recálculo de las físicas 60 veces por segundo.

Al usar un *Compute Shader* esta carga de trabajo se realiza de forma paralela permitiendo tener la resolución en $384 \times 120 \times 384$ vóxeles, asumiendo los 12 segundos de inicialización, garantizando un buen equilibrio entre resolución geométrica y altas tasas de fotogramas (176 FPS).

Experimento 2: Rendimiento sostenido frente a la carga dinámica

Una vez decidida que la resolución ideal que se va a usar es $384 \times 120 \times 384$ vóxeles, este segundo experimento se encarga de evaluar la estabilidad de los FPS durante la ejecución completa de un incendio, desde el inicio del foco hasta que quede completamente cubierto de llamas el mapa. El propósito principal es evaluar cómo le afecta al rendimiento el incremento de voxels con fuego.

La metodología que se ha seguido ha consistido en encender un foco en el centro y dejar que se propague de manera libre, sin viento ni humedad, a través de todo el mapa con las mismas condiciones que los anteriores (sin limitaciones de fotogramas y con vegetación continua). Se han registrado los datos de FPS cada 4 segundos desde el segundo 0, justo antes de iniciar el incendio, hasta que se haya propagado y cubierto el mapa por completo.

Tiempo (s)	Estado Visual del Incendio	Rendimiento (FPS)
0	Reposo (Sin ignición)	172
4	Foco central en expansión	164
8	Crecimiento radial acelerado	169
12	Cobertura media del mapa	180
16	Incendio masivo (Casi total)	196
20	Mapa crítico (Llegando a bordes)	204
24	Estabilización (Totalmente calcinado)	210
28	Estabilización final	210

Tabla 4.2: Evolución dinámica del rendimiento durante la propagación

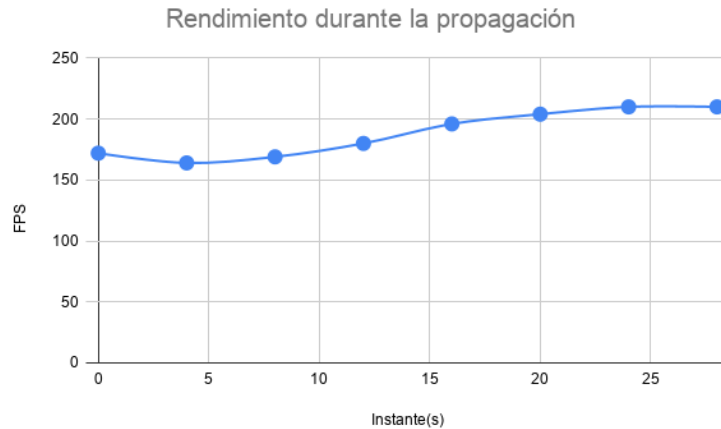


Figura 4.6: Evolución de Fotogramas por Segundo (FPS) durante la expansión térmica.

Lejos de sufrir una caída de fotogramas a medida que el fuego cubre todo el mapa como se hubiese esperado, en *Tabla 4.2* y *Figura 4.6* se observa un ligero descenso del rendimiento en los primeros instantes, momento en el que el frente de es más activo y la GPU calcula un mayor número de probabilidades, para después volver a subir.

A partir del segundo 8, la tasa de fotogramas comienza a subir hasta estabilizarse en 210 FPS. Este incremento del rendimiento, mayor que en el estado de reposo inicial, válida las dos reglas de optimización programadas en la arquitectura visual y física:

1. **Cese computacional (Compute Shader):** Al implementar una validación que salta las celdas cuyo estado es ya “En llamas”, la GPU deja de calcular las matemáticas complejas de las probabilidades en las áreas ya quemadas del interior de la matriz.
2. **Oclusión volumétrica (Raymarching):** El algoritmo de trazado de rayos detiene su avance en el momento en el que la densidad del humo acumulado alcanza el valor de 1.0 (opacidad total) ahorrando así millones de lecturas internas cuanto más porcentaje de mapa quemado haya

Capítulo 5

Conclusiones

En este Trabajo de Fin de Grado se presenta como objetivo el desarrollo de un simulador interactivo de incendios forestales desarrollado en Godot 4 que unifica el rigor termodinámico científico con la inmersión visual de los motores gráficos. Este objetivo se ha logrado mediante el diseño de una arquitectura de software basada en la computación masivamente paralela (GPGPU) en la que por un lado, se ha implementado la lógica de Autómatas Celulares para calcular la propagación física del fuego mientras que por el otro se ha logrado una representación realista usando algoritmos de Ray-marching volumétrico sumados a mapas de elevaciones para la topografía. La sincronización de ambos sistemas ha dado como resultado una herramienta interactiva capaz de simular y visualizar la propagación del fuego con rigor físico y un alto rendimiento computacional.

A continuación se detalla el grado en el que se han cumplido los objetivos planteados al inicio de este proyecto:

1. **Investigar y procesar datos geográficos reales para la generación de la topografía 3D:** Se ha establecido un flujo de trabajo para la obtención de Modelos Digitales de Elevación (como los provistos por el CNIG). Mediante el uso del software de sistemas de información geográfica **QGIS**, estos datos en bruto han sido procesados, dimensionados y convertidos a mapas de alturas (*heightmaps*) normalizados permitiendo su correcta importación y reconstrucción tridimensional dentro del motor gráfico.
2. **Dominar el entorno de Godot 4, GDScript y su lenguaje de *shaders* visuales:** Se ha dominado la arquitectura de nodos del motor, desarrollando scripts robustos en GDScript para orquestrar el ciclo de vida de la aplicación, y el lenguaje de *shaders* imprescindible para la manipulación a bajo nivel del *pipeline* gráfico.

3. **Investigar e integrar *Compute Shaders* para cálculos de propósito general (GPGPU):** Ha supuesto una de las mayores complicaciones técnicas del proyecto pero se ha implementado con éxito el uso de la API `RenderingDevice` de Godot, estableciendo una comunicación segura y sincronizada mediante *buffers* de memoria entre la CPU y la GPU para delegar la carga algorítmica masiva.
4. **Implementar la lógica matemática de propagación mediante autómatas celulares:** El *Compute Shader* evalúa iterativamente el estado de cada celda, aplicando las ecuaciones termodinámicas de Rothermel de manera paralela en cada fotograma, logrando un comportamiento orgánico y físicamente fundamentado.
5. **Diseñar e implementar el renderizado volumétrico mediante *Raymarching*:** Se ha programado un *shader* espacial que ejecuta el algoritmo de *Raymarching*. El cual se encarga de leer los datos térmicos generados por el autómata celular y renderiza el fuego volumétrico en tiempo real.
6. **Integrar los sistemas en un entorno interactivo y controlable por el usuario:** Se ha desarrollado una Interfaz de Usuario (UI) funcional y modular en la cual el usuario puede inyectar focos de ignición mediante el puntero y alterar las variables medioambientales críticas (velocidad y dirección del viento, nivel de humedad) observando la reacción del modelo de forma inmediata.
7. **Evaluar el rendimiento computacional y la estabilidad de la simulación:** Las pruebas de estrés realizadas demuestran la eficiencia del simulador permitiendo mantener tasas de fotogramas (FPS) estables con grandes resoluciones.

5.1. Posibles líneas y mejoras futuras de trabajo

La arquitectura desarrollada en este proyecto establece unas bases sólidas para realizar diversas expansiones y optimizaciones técnicas en un futuro. Se proponen las siguientes mejoras:

- **Implementación y dinámica volumétrica del humo:** Debido a las restricciones temporales en el desarrollo de este Trabajo de Fin de Grado, el modelo visual se ha centrado en el avance directo del frente de llamas. Una mejora prioritaria consistiría en ampliar los *buffers* del *Compute Shader* para emitir densidad de partículas y extender el *Raymarching* para renderizar la dispersión y el auto-sombreado de las columnas de humo, incrementando drásticamente el realismo y el valor táctico de la herramienta.

- **Modelos ópticos de iluminación volumétrica:** Una mejora natural consistiría en refinar la interacción lumínica en el interior del fluido. Se propone implementar un modelo matemático basado en emisión y absorción dentro del *shader*, lo que permitiría vincular la densidad computada para modular la emisión térmica del fuego, y simultáneamente aplicar la ley de Beer-Lambert para calcular la absorción lumínica del humo.
- **Implementación completa del Método de Monte Carlo:** Aprovechando el rendimiento del *Compute Shader*, se podría adaptar el simulador para ejecutar múltiples instancias del incendio simultáneamente, generando un mapa de calor que muestre las probabilidades de que se queme cada voxel del terreno.
- **Integración de Datos Vectoriales GIS y Tipos de Combustible:** Modificar el sistema para que admita múltiples capas de textura (*Splatmaps*), permitiendo leer la densidad y el tipo de vegetación real (pinar, matorral, pasto) de fuentes satelitales, alterando dinámicamente la variable de Probabilidad Base (P_0) en distintas zonas del terreno.
- **Simulación de focos secundarios (Pavesas):** Incorporar un modelo para calcular el movimiento de pavesas propulsadas por el viento que inicien nuevos focos, con esto se lograría aumentar el realismo haciendo que el fuego pueda llegar a cruzar barreras como ríos o cortafuegos y que no se pare en seco.

Anexos

En este anexo se detallan las instrucciones necesarias para acceder al código, así como los requisitos técnicos para compilar y ejecutar el simulador gráfico de incendios forestales de forma local.

Acceso al Código Fuente

El proyecto está versionado utilizando el sistema Git y alojado en la plataforma GitHub. El repositorio incluye tanto el código fuente de Godot (GDScript) como los *Shaders* de la GPU para el correcto funcionamiento del Autómata Celular y el *Raymarching*.

- **Enlace al repositorio:** <https://github.com/GerardoBonet25/TFG>
- **Rama principal:** `main` (contiene la versión estable documentada en esta memoria).

En este directorio se encuentran los archivos tanto del simulador principal (SimuladorIncendios) como los archivos de la memoria y un pequeño simulador inicial 2D que realicé para comprender las bases (InicioPropagacion2D).

Requisitos del Sistema

Debido al uso de tecnologías de renderizado avanzado y paralelización masiva en GPU, el equipo informático destinado a ejecutar la simulación debe cumplir con los siguientes requisitos mínimos:

- **Sistema Operativo:** Windows 10/11.
- **Motor Gráfico:** Godot Engine versión 4.2 o superior.
- **API Gráfica:** Tarjeta gráfica (GPU) dedicada o integrada con soporte completo para **Vulkan 1.2**. Este requisito es fundamental ya que la API utilizada para despachar los *Compute Shaders* del Autómata Celular no es compatible con versiones antiguas de OpenGL.

Bibliografía

- [1] J. Hughes, *Computer Graphics: Principles and Practice*, ser. The systems programming series. Addison-Wesley, 2014. [Online]. Available: <https://books.google.es/books?id=OVpsAQAAQBAJ>
- [2] QGIS Development Team, *QGIS Geographic Information System*, Open Source Geospatial Foundation, 2024, procesamiento SIG, normalización y adecuación de MDE. [Online]. Available: <https://qgis.org>
- [3] Centro Nacional de Información Geográfica (CNIG), “Centro de descargas del cnig: Mapas de elevaciones y nubes de puntos lidar,” 2025. [Online]. Available: <https://centrodedescargas.cnig.es>
- [4] R. C. Rothermel, “A mathematical model for predicting fire spread in wildland fuels,” USDA Forest Service, Intermountain Forest and Range Experiment Station, Ogden, UT, Tech. Rep. INT-115, 1972.
- [5] P. Enders, “Huygens’ principle as universal model of propagation,” *Latin-American Journal of Physics Education*, vol. 3, no. 1, p. 4, 2009.
- [6] K. G. Hirsch, “Canadian forest fire behavior prediction system,” Canadian Forest Service Publications, Ottawa, Canada, Tech. Rep., 1996.
- [7] M. Wotton, “Interpreting and using outputs from the canadian forest fire danger rating system in research applications,” *Environmental and Ecological Statistics*, vol. 16, pp. 107–131, 06 2009.
- [8] L. P. Kobbelt, S. Bischoff, M. Botsch, K. Kähler, C. Rössl, R. Schneider, and J. Vorsatz, “Geometric modeling based on polygonal meshes,” Max-Planck-Institut für Informatik, Tech. Rep. MPI-I-2000-4-002, 2000. [Online]. Available: https://pure.mpg.de/rest/items/item_1819314/component/file_3662022/content
- [9] A. E. Kaufman and K. Mueller, “Overview of volume rendering.” *The visualization handbook*, vol. 7, pp. 127–174, 2005.
- [10] M. Zwicker, H. Pfister, J. Van Baar, and M. Gross, “Ewa volume splatting,” in *Proceedings Visualization, 2001. VIS’01.* IEEE, 2001, pp. 29–538.

- [11] J. J. Caban and P. Rheingans, "Texture-based transfer functions for direct volume rendering," *IEEE Transactions on Visualization and Computer Graphics*, vol. 14, no. 6, pp. 1364–1371, 2008.
- [12] J. T. Kajiya and B. P. Von Herzen, "Ray tracing volume densities," in *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '84. Association for Computing Machinery, 1984, p. 165–174.
- [13] M. J. Harris, "Chapter 38. fast fluid dynamics simulation on the gpu," in *GPU Gems: Programming Techniques, Tips and Tricks for Real-Time Graphics*. Addison-Wesley Professional, 2004. [Online]. Available: <https://developer.nvidia.com/gpugems/gpugems/part-vi-beyond-triangles/chapter-38-fast-fluid-dynamics-simulation-gpu>
- [14] K. Crane, I. Llamas, and S. Tariq, "Chapter 30. real-time simulation and rendering of 3d fluids," in *GPU Gems 3*. Addison-Wesley Professional, 2007. [Online]. Available: <https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-30-real-time-simulation-and-rendering-3d-fluids>
- [15] T. Davidovič, T. Engelhardt, I. Georgiev, P. Slusallek, and C. Dachsbacher, "3d rasterization: a bridge between rasterization and ray casting," in *Proceedings of Graphics Interface 2012*, 2012, pp. 201–208. [Online]. Available: <http://iliyan.com/publications/3DRasterization/3DRasterization-GI2012.pdf>
- [16] N. Max, "Optical models for direct volume rendering," *IEEE Transactions on visualization and computer graphics*, vol. 1, no. 2, pp. 99–108, 2002.
- [17] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3d surface construction algorithm," in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '87. Association for Computing Machinery, 1987, p. 163–169.
- [18] M. J. M.-F. Guisado-Lizar and P. Real, "Versatilidad y eficiencia de los autómatas celulares para la simulación y análisis de sistemas complejos en ingeniería."
- [19] F. Rodríguez y Silva, "Sistema uco40: Catálogo de modelos de combustible forestal," Laboratorio de Defensa contra Incendios Forestales (LABIF-UCO), Universidad de Córdoba.
- [20] J. M. F. Oro, *Técnicas numéricas en ingeniería de fluidos: introducción a la dinámica de fluidos computacional (CFD) por el método de volúmenes finitos*. Reverté, 2012.

-
- [21] A. Alexandridis, D. Vakalis, C. I. Zelilidou, and B. Boutsinas, “A cellular automata model for forest fire spread prediction: The case of the wildfire that swept through spetses island in 1990,” *Applied Mathematics and Computation*, vol. 204, no. 1, pp. 191–201, 2008. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0096300308004943>
- [22] Y. Carmel, S. Paz, F. Jahashan, and M. Shoshany, “Assessing fire risk using monte carlo simulations of fire spread,” *Forest Ecology and Management*, vol. 257, no. 1, pp. 370–377, 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0378112708006919>
- [23] “A monte carlo approach to assessing the uncertainty of wildfire spread simulations,” *Fire*, vol. 3, no. 4, p. 71, 2020. [Online]. Available: <https://www.mdpi.com/2571-6255/3/4/71>
- [24] D. Escudero-Mancebo, *Apuntes de Informática Gráfica*. Universidad de Valladolid, 2023, pp. 1–3, edición 2023. [Online]. Available: <https://uvadoc.uva.es/bitstream/handle/10324/59563/IG.pdf?sequence=1>
- [25] J. Linietsky, A. Manzur, and Godot Engine Contributors, “Godot engine,” 2024, motor gráfico de código abierto utilizado para la arquitectura híbrida. [Online]. Available: <https://godotengine.org>
- [26] A. Bagdasaryan, “Discrete dynamic simulation models and technique for complex control systems,” *Simulation Modelling Practice and Theory*, vol. 19, no. 4, pp. 1061–1087, 2011.
- [27] R. D. Zinck, K. Johst, and V. Grimm, “Wildfire, landscape diversity and the drossel–schwabl model,” *Ecological Modelling*, vol. 221, no. 1, pp. 98–105, 2010.
- [28] I. Karafyllidis and A. Thanailakis, “A model for predicting forest fire spreading using cellular automata,” *Ecological Modelling*, vol. 99, no. 1, pp. 87–97, 1997.
- [29] M. Schütz, M. Bernhard, and M. Wimmer, “Rendering point clouds with compute shaders and vertex order optimization,” *Computer Graphics Forum*, vol. 40, no. 2, pp. 115–126, 2021. [Online]. Available: <https://onlinelibrary.wiley.com/doi/full/10.1111/cgf.14345>
- [30] D. Gallup, M. Pollefeys, and J.-M. Frahm, “3d reconstruction using an n-layer heightmap,” in *Joint Pattern Recognition Symposium*. Springer, 2010, pp. 1–10.